TÉCNICO
UNIVERSIDADE
DE LISBOA



DEPT: Defense via Ensemble and Purification Transformer

Guilherme Silvério de Carvalho Romeiro Leitão

Thesis to obtain the Master of Science Degree in

Computer Science and Engineering

Supervisor: Prof. Fernando Manuel Valente Ramos

October 2026

Declaration
I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa.

Acknowledgments

I would like to express my gratitude to all those who have contributed to the completion of this thesis. Their support, guidance, and encouragement have been invaluable throughout this journey.

First, to my supervisor, Prof. Fernando Ramos, for his guidance throughout this thesis, for the questions that sent me back to the drawing board when I needed them, and for the trust he placed in this work.

To my girlfriend Carolina, for her patience during the long months this thesis took, for offering her attention more times than I can count, for being the light at the end of the tunnel, and for her love and support throughout this journey. Thank you gorgeous.

To my father, for his encouragement, for always being ready to listen to my endeavors and countless brainstorming sessions, for his presence during the long hours of work, and for his continuous unwavering support in everything I do.

To my mother, for her love, for her attention, for her constant care and concern, and for always being there for me, without ever asking for anything in return. Together with Rui, thank you both for your presence and for always making sure I had everything I needed to succeed.

Finally, to Inês, whose work on TANTGen this thesis is built upon, for the clarifications, for the materials that made this work possible, and without whom this thesis would not have even existed.

To each and every one of you — Thank you so much.

Abstract

Machine-learning-based network intrusion detectors can be evaded by adversaries who subtly manipulate network traffic (e.g., by changing packet sizes or inter-packet arrival times), while preserving protocol compliance and the ability to achieve their attack objectives. However, many state-of-the-art defenses are evaluated using feature perturbations that may not translate into realizable network traffic, limiting their practical applicability. This thesis proposes DEPT, a streaming defense that detects and purifies manipulated traffic before it reaches a protected detector.

DEPT represents traffic as masked packet-level bits and flow statistics at two temporal granularities. For each packet, an attention-based meta-learner combines a Random Forest and a Temporal Convolutional Network to identify manipulated flows. Flagged flows are corrected in feature space using a causal purification Transformer, with the corrected features mapped back to packet sizes and timings. . Training and evaluation rely on TANTGen [1], a black-box adversarial traffic generator, covering ten traffic sources from three public datasets and three perturbation modes.

Experimental results show that DEPT achieves an end-of-flow AUROC of 0.933 in detecting adversarial manipulations. Although the Random Forest alone outperforms the ensemble from the second packet onwards, the ensemble enables detection from the very first packet and provides greater robustness. Purification restores 63% of the detection lost to adversarial manipulation, reducing attack success from 21.1% to 11.6%. In contrast, a state-of-the-art diffusion-based purifier [2] recovers almost none of the lost detection. Moreover, 97.2% of DEPT's generated packets parse without malformed-packet errors, supporting the practical feasibility of the approach. Nevertheless, performance varies across traffic source, with manipulation detection struggling on size-perturbed reflection floods, and purification occasionally reducing detection performance on flows that had not successfully evaded the detector.

Keywords

Adversarial Machine Learning; Network Intrusion Detection; Evasion Attacks; Ensemble Learning; Adversarial Purification; Streaming Traffic Analysis

Resumo

Os detetores de intrusões em rede baseados em ML podem ser evadidos alterando apenas os tempos e tamanhos dos pacotes, sem quebrar o protocolo nem o ataque. Contudo, muitas defesas são avaliadas com perturbações no feature-space que podem não ser realizáveis. Propõe-se o DEPT, uma defesa em tempo real que deteta e purifica o tráfego manipulado antes do detetor protegido.

O tráfego é representado por bits mascarados de cada pacote e estatísticas de fluxo. A cada pacote, um attention-based meta-learner combina uma *Random Forest* e uma *Temporal Convolutional Network* para detetar fluxos manipulados, que um Transformer causal corrige em feature-space antes da projeção para traffic-space.

O DEPT é treinado e avaliado com tráfego adversarial do gerador black-box TANTGen [1]: dez fontes de três conjuntos de dados, em três modos de perturbação. A deteção de manipulação atinge uma AUROC final de 0,933, abaixo da Random Forest isolada desde o segundo pacote, mas o *ensemble* prevê já no primeiro e resiste melhor aos ataques testados contra o modelo sequencial. No detetor independente visado por cada ataque, a purificação recupera 63% da deteção perdida. No sistema completo, o sucesso dos ataques cai de 21,1% para 11,6%, enquanto um purificador por difusão [2] quase nada recupera. Dos pacotes realizáveis, 97,2% são descodificados sem erros de malformação. A eficácia varia por fonte: a deteção fica próxima ou abaixo do acaso nos reflection-floods com perturbação de tamanho, e purificar fluxos que não evadiram pode reduzir a deteção a jusante.

Palavras Chave

Aprendizagem Automática Adversarial; Deteção de Intrusões em Rede; Ataques de Evasão; Métodos de Ensemble; Purificação de Tráfego; Análise de Tráfego em Tempo Real

Contents

1 Introduction **1**

1.1 Context and Motivation . . . 2

1.2 Problem Statement . . . 3

1.3 Work Objectives and Challenges . . . 4

1.4 Rationale and Contributions . . . 4

1.5 Organization of the Document . . . 6

2 Background **7**

2.1 Machine Learning for NIDS . . . 7

2.1.1 Learning Paradigms . . . 8

2.1.2 Feature Engineering and Representation . . . 8

2.1.3 Model Architectures Relevant to DEPT . . . 10

2.2 Adversarial Machine Learning . . . 11

2.2.1 Threat Taxonomy . . . 12

2.2.2 Defense Paradigms . . . 12

2.2.3 Realizability and Protocol Constraints . . . 13

2.3 Ensemble Learning for Robustness . . . 14

2.3.1 Bagging, Boosting, Stacking . . . 14

2.3.2 Gradient Masking and Transferability . . . 15

2.4 Summary . . . 16

3 Related Work **17**

3.1 Adversarial Attacks against ML-based NIDS . . . 17

A – Feature-space attacks. . . . 18

B – Traffic-space attacks. . . . 18

3.1.1 TANTGen, the Attacker Used in This Work . . . 18

3.2 Training-time Defenses . . . 19

A – Data sanitization and poisoning defenses. . . . 19

B – Federated and backdoor defenses. . . . 19

C – Ensemble-based hardening. 20
D – Realizable adversarial training. 20
3.3 Inference-time Defenses 20
A – Certified robustness. 21
B – Generative purification. 21
C – Constraint-based and auxiliary detection. 21
3.4 Positioning of DEPT and Gap Analysis 22
4 Solution: DEPT Architecture **25**
4.1 Threat Model 26
4.2 System Overview 27
4.3 Dataset Construction and Packet-to-Tensor Pipeline 29
A – Traffic corpus. 29
B – Label convention. 29
C – From packets to tensors. 30
D – Splits and class balance. 31
4.4 Primary Detection: Heterogeneous Stacking Ensemble 32
4.4.1 Random Forest Base Learner 32
4.4.2 Temporal Convolutional Network Base Learner 32
4.4.3 Meta-Learner (the “Mini-Transformer”) 33
4.5 Input Purification: Reconstructor 35
4.5.1 Purification Model 35
4.5.2 Training on Matched Flow Pairs 36
4.5.3 Traffic-Space Realization and Validation 36
4.6 Implementation and Tools 37
5 Evaluation **39**
5.1 Research Questions 40
5.2 Experimental Setup 40
5.3 Metrics 41
A – Detection. 41
B – Purification. 41
5.4 Detection Results 41
5.4.1 Streaming Detection 41
5.4.2 Detection per Source and Mode 42
5.4.3 One Global Model or Ten Specialists? 43
5.5 Robustness under a Gradient-guided Attacker (RQ1) 45

5.5.1 Attack Setup . . . 45
5.5.2 Results . . . 46
A – The stack is the hardest target at low budgets. . . . 46
B – The meta-learner leans on the forest under attack. . . . 47
5.6 Purification Results . . . 47
5.7 End-to-End Evaluation . . . 49
5.7.1 Detect-then-Purify Pipeline . . . 49
5.7.2 Comparison with a Published Purifier . . . 52
A – Where TANTGen evaded, DEPT recovers detection and the diffusion purifier does not. . . . 53
B – Elsewhere, both purifiers can lower detection. . . . 53
6 Conclusion **55**
6.1 Summary of Contributions . . . 55
6.2 Answers to Research Questions . . . 56
6.3 System Limitations . . . 57
6.4 Future Work . . . 58
Bibliography **59**

List of Figures

1.1 The adversarial evasion problem. 2

2.1 The two supervision regimes most relevant to DEPT. 9
2.2 The one-dimensional convolutional pipeline. 10
2.3 The decoder-only Transformer block. 11
2.4 Purification as projection onto the benign manifold. 13
2.5 The three classical ensemble strategies. 15

3.1 The surveyed defenses by strategy, phase and space. 22

4.1 DEPT's deployment context. 26
4.2 The DEPT architecture. 28
4.3 The nPrint per-packet representation with identity masking. 30
4.4 The dual prediction cadence over a flow. 31
4.5 Dilated causal convolutions and the TCN receptive field. 33
4.6 The purification stage. 35

5.1 Streaming AUROC@t. 42
5.2 Per-source AUROC distribution. 44
5.3 Evasion cost: one perturbation, three targets. 46
5.4 A single real packet through the purification pipeline. 50
5.5 Deployment baseline: NIDS attack success before vs. after DEPT. 51
5.6 DEPT's purifier against the diffusion purifier, per source and mode. 54

List of Tables

3.1 Comparison of the surveyed adversarial defenses. 23

4.1 The two representations DEPT derives from the same packets. 31
4.2 The 13-dimensional fusion token. 34
4.3 Model configuration summary. 38

5.1 Streaming detection 42
5.2 Per-source AUROC 43
5.3 Global model against per-attack specialists 44
5.4 Evasion cost under a gradient-guided attacker. 46
5.5 Purification results (overall) 47
5.6 Purification by source 48
5.7 Deployment baseline: attack success before vs. after DEPT 52
5.8 Comparison with the diffusion purifier of Merzouk et al. 53

Acronyms

AML Adversarial Machine Learning

ART Adversarial Robustness Toolbox

ASR Attack Success Rate

AUROC Area Under the Receiver Operating Characteristic curve

CNN Convolutional Neural Network

DDoS Distributed Denial of Service

EDP Earliest Detection Packet

FGSM Fast Gradient Sign Method

FL Federated Learning

GAN Generative Adversarial Network

ICMP Internet Control Message Protocol

IFM Inverse Feature Mapping

IP Internet Protocol

IPAT Inter-Packet Arrival Time

MDP Markov Decision Process

ML Machine Learning

MSE Mean Squared Error

MTU Maximum Transmission Unit

NIDS Network Intrusion Detection System

PGD Projected Gradient Descent

RF Random Forest

RL Reinforcement Learning

SMT Satisfiability Modulo Theories

TCN Temporal Convolutional Network

TCP Transmission Control Protocol

TEE Trusted Execution Environment

TTL Time-to-Live

UDP User Datagram Protocol

1

Introduction

Contents

1.1 Context and Motivation 2
1.2 Problem Statement 3
1.3 Work Objectives and Challenges 4
1.4 Rationale and Contributions 4
1.5 Organization of the Document 6

Machine learning has become an important approach to network intrusion detection. Signature-based detectors rely on known attack patterns, which limits their ability to detect new attacks or variants that do not match existing signatures. They can also be evaded by modifying traffic to avoid these patterns while preserving attack functionality. Learned models can generalize beyond known signatures, but introduce a different attack surface: attackers can deliberately manipulate traffic features to exploit the models' learned decision boundaries. This thesis addresses such adversarial evasion through changes to packet timing and size that preserve protocol compliance and attack functionality, and investigates how to defend against it. It proposes DEPT (Defense via Ensemble and Purification Transformer), a streaming defense that detects adversarially manipulated network traffic and aims to reverse the manipulation before it reaches the detector it was meant to evade.

1.1 Context and Motivation

A Network Intrusion Detection System (NIDS) monitors network traffic to identify malicious activity. Traditionally, detection has relied on matching traffic against signatures of known threats, which can accurately identify known attacks but may miss new attacks or variants that do not match existing signatures. Machine Learning (ML) offers a different approach: instead of explicitly specifying attack patterns, a model learns the statistical behaviour that separates malicious from benign traffic, and can therefore generalize to variants and previously unseen attacks [3, 4]. Learning-based detection is a major research direction in the field and an increasingly common deployment choice [5, 6].

This shift introduced a new attack surface. A learned model can be fooled: a small, deliberate perturbation of its input may cause a misclassification. An input crafted to induce such an error is known as adversarial example [7, 8]. For a NIDS, adversarial examples can enable evasion: the attacker reshapes ongoing attack traffic just enough for the detector to label it benign, while preserving the attack's functionality [9]. This threat is not hypothetical. Prior work has demonstrated that black-box attacks can evade deep-learning-based intrusion detectors by manipulating traffic features [10]. Figure 1.1 illustrates the threat addressed in this thesis.

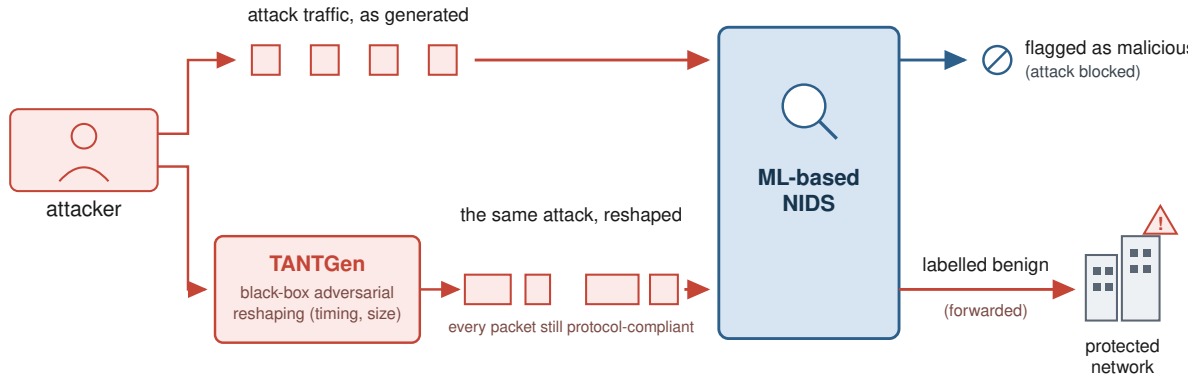


Figure 1.1: The problem this thesis addresses. The original attack traffic is flagged by the ML-based NIDS that guards the network (top path). A black-box generator, TANTGen [1] in this work, reshapes the same attack, altering only packet timing and sizes while preserving protocol compliance and attack functionality. The modified traffic is then classified as benign, allowing the attack to evade detection (bottom path). DEPT aims to detect and reverse these manipulations, restoring the NIDS's ability to identify the attack.

Evaluating adversarial robustness requires accounting for the constraints of real network traffic. In the image domain, where adversarial examples were first studied, a perturbed pixel array can still be interpreted as an image; a perturbed feature vector of network traffic may not correspond to any realizable packet sequence, since packets must keep their checksums, lengths, and protocol semantics consistent [11, 12]. In most practical scenarios, an attacker operates in the black-box setting: it can probe a deployed detector and observe outcomes, but has access to neither the model's architecture nor its gradients [13, 14]. Much of the defensive literature sets aside one or both of these constraints, so

its robustness results may not reflect threats feasible under these conditions; Chapter 3 examines this mismatch.

This work, together with a companion thesis, addresses this mismatch. TANTGen [1] addresses the realism gap on the attack side: it is a Transformer-based generator that manipulates attack traffic under black-box constraints, altering only packet timing and packet size, and the traffic it emits is protocol-compliant by construction. DEPT provides the defense side of the same effort. It is trained on, and evaluated against, the realizable adversarial traffic that TANTGen produces, grounding the evaluation in packet-level attacks rather than feature perturbations whose realizability has not been established. Only this traffic is used, not TANTGen itself, so another generator with the same constraints could take its place.

1.2 Problem Statement

This thesis addresses the following problem. A stream of flows arrives at the boundary of a protected network, where a NIDS is deployed. Part of the traffic is benign; part belongs to conventional attacks, which the deployed NIDS is assumed to detect; and part consists of attack traffic that has been adversarially manipulated to evade detection. The defense must decide, for each flow and while the flow is still in progress, whether it has been manipulated, and, when it has, counteract the manipulation before the protected NIDS processes the traffic.

Three constraints define an acceptable solution. First, streaming operation: the verdict must be produced packet by packet, as the flow unfolds, because waiting for a flow to complete may allow an attack to succeed before detection, while buffering entire flows can impose excessive memory and latency costs. Second, realistic attack assumptions: the defense must be trained and evaluated against realizable, protocol-compliant adversarial traffic generated under the assumed black-box access constraints, rather than unconstrained feature-space perturbations. Third, realizable output: a flow the defense repairs must remain valid network traffic, preserving protocol semantics as well as packet-level correctness; otherwise the defense has merely moved the realizability problem from the attacker to the defender.

Two scope limitations complete the problem statement. DEPT complements the protected NIDS, it does not replace it: recognizing the underlying attacks remains the downstream system's task, while DEPT detects and attempts to reverse adversarial manipulation of the traffic. Since the defense is designed to operate inline, its processing overhead must also meet real-time latency requirements; evaluating the implemented system against those requirements is left to future work (Section 6.4).

1.3 Work Objectives and Challenges

The problem statement is divided into four objectives, which structure both the design presented in Chapter 4 and the research questions that guide the evaluation of Chapter 5:

- **O1 (traffic representation).** Develop a representation of raw traffic that preserves sufficient packet-level information for model outputs to be mapped back to realizable traffic, and that architecturally different learners can consume at different temporal granularities, from individual packets to aggregated flow statistics.
- **O2 (heterogeneous detection).** Build a detection ensemble whose members belong to different architectural families, with the aim of reducing the transferability of adversarial perturbations between them, and combine their predictions into a single per-packet verdict.
- **O3 (input purification).** Learn to reconstruct a flagged flow: remove the adversarial manipulation and restore the flow's characteristics before manipulation, while preserving a valid, protocol-compliant packet sequence.
- **O4 (streaming-aware evaluation).** Evaluate the system under streaming conditions: on partial flows as they arrive, with metrics that measure how early a verdict becomes reliable, rather than end-of-flow accuracy alone.

Three challenges arise in this setting. The first is imbalance: the corpus used in this work is dominated by high-rate floods, so a handful of traffic sources contributes the overwhelming majority of flows. Training and evaluation must account for this imbalance to avoid under-representing other sources and obscuring poor performance on them on aggregate results. The second is the limited observable signal: a manipulation confined to packet timing on a high-rate flood may produce only small changes in the observed features, making manipulated and unmodified traffic difficult to distinguish; Chapter 5 quantifies this limitation. The third is that the defense shares the attacker's realizability constraint: purification edits traffic, and edited traffic must remain realizable, which rules out treating flows as unconstrained feature vectors and informs the representation choices of O1. All three challenges must also be addressed within the latency requirements of streaming operation.

1.4 Rationale and Contributions

The design of DEPT rests on two decisions, motivated briefly here and discussed in more detail in Chapters 2 and 3. The first concerns detection. One strategy available to a black-box attacker exploits transferability: perturbations crafted against a substitute model may also evade the target model [13]. DEPT's

detector therefore stacks two learners from different architectural families, a Random Forest (RF), which is not directly amenable to gradient-based optimization, and a Temporal Convolutional Network (TCN), which processes packet-level feature sequences. A small attention-based meta-learner combines their predictions at every packet [15]. This architectural diversity aims to reduce transferability: a perturbation that changes one learner's prediction may not change the other's.

The second decision concerns what to do with a flagged flow. Dropping it would risk denying service to legitimate clients whose traffic is incorrectly flagged, so DEPT instead attempts to remove the manipulation through input purification, an approach explored in works such as Defense-GAN [16]. The purifier is a causal Transformer that predicts a residual correction over the packet sequence, rather than a Generative Adversarial Network (GAN) or a diffusion model: self-attention captures dependencies between packets, causality allows each correction to depend only on the current and preceding packets, and a residual head allows the input to remain unchanged when the predicted correction is zero. Section 5.7.2 compares this choice with a published diffusion-based purifier.

Building on this rationale, the thesis makes five contributions:

- A packet-to-tensor pipeline that turns raw captures into masked, bit-level packet representations and aggregate flow statistics, so that architecturally different learners may process the same traffic at different temporal granularities (Section 4.3).
- A heterogeneous detection ensemble comprising an RF and a TCN, whose predictions are combined per packet by an attention-based meta-learner, together with a streaming comparison that evaluates every model by how early, and not only how well, it separates manipulated from unmodified flows (Sections 4.4 and 5.4.1).
- A causal purification Transformer coupled to an explicit realization step: flagged flows are corrected in feature space, the corrections are applied to packet timing and size, and the resulting packets are checked for parsing errors with a standard protocol dissector (`tshark`) (Sections 4.5 and 5.6).
- An end-to-end evaluation that measures DEPT as a defense placed in front of a protected NIDS: unmodified, TANTGen-perturbed, and DEPT-defended traffic are each evaluated by the independent NIDS targeted by the attack, reporting changes in the attack success and detection rates, and DEPT's purifier is compared with a published diffusion-based purifier placed in the same position (Section 5.7).
- An adversarial evaluation corpus of ten traffic sources drawn from three public datasets (Kitsune, CIC-IDS2017, and CIC-DDoS-2019 [17–19]), each manipulated by TANTGen in three perturbation modes, with a per-source analysis of where the defense is effective, where it fails, and the factors underlying these outcomes (Section 4.3 and Chapter 5).

1.5 Organization of the Document

The remainder of this document is organized as follows. Chapter 2 presents the background: learning-based intrusion detection and traffic representation, adversarial machine learning and the realizability constraints that distinguish it in the network domain, and the ensemble concepts behind DEPT's detector. Chapter 3 surveys related work on the attack and the defense sides, and states the gap DEPT fills. Chapter 4 presents DEPT: the threat model, a system overview, the corpus and packet-to-tensor pipeline, the detection ensemble, and the purification and realization stages. Chapter 5 evaluates the proposed system. Chapter 6 closes with the answers to the research questions, the system's limitations, and future work.

2

Background

Contents

2.1 Machine Learning for NIDS 7
2.2 Adversarial Machine Learning 11
2.3 Ensemble Learning for Robustness 14
2.4 Summary 16

DEPT combines three main ingredients: a bit-level representation of network traffic, a heterogeneous detection ensemble, and a learned purifier. This chapter presents the concepts each of them builds upon. Instead of a broad survey of machine learning for security, the focus is kept on what the system relies on, so that the design decisions of Chapter 4 follow from the approaches, techniques, and principles introduced here.

2.1 Machine Learning for NIDS

A NIDS inspects network traffic in order to separate benign activity from attacks. Classical systems match traffic against hand-written signatures of known threats, which makes them precise on catalogued

attacks but unable to detect anything absent from the rule set, including minor variants of existing malware. ML reformulates detection as a data-driven problem: instead of enumerating signatures, a model learns the statistical structure that separates benign from malicious traffic, and can therefore generalize to variants and to previously unseen attacks [3, 4]. This shift supports a large body of modern detection research [5, 6], and it defines the setting in which DEPT operates.

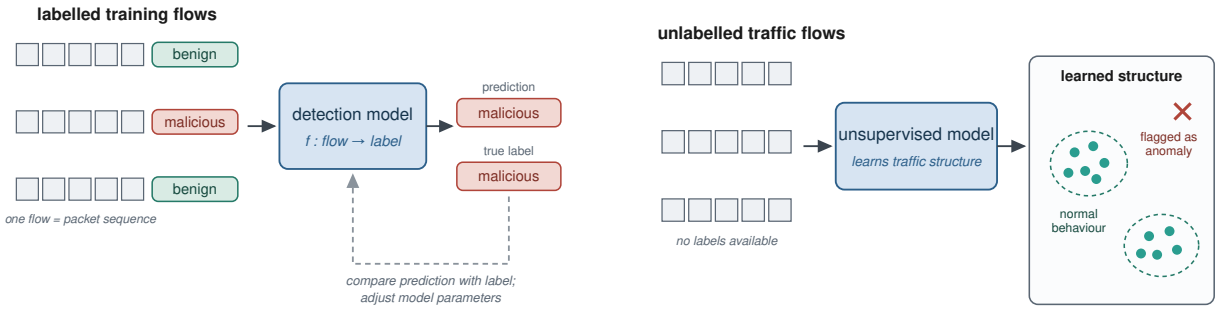
2.1.1 Learning Paradigms

Learning-based intrusion detection is usually organized by the kind of supervision a model requires.

Supervised learning trains on labelled data, where each example is marked benign or malicious, and learns a mapping from input features to labels (Figure 2.1). Random forests and support vector machines follow this paradigm, as do deep neural networks when trained as classifiers [3, 20]; a neural network is an architecture rather than a paradigm, however, and can equally be trained without labels, as the autoencoders discussed below are. Every component of DEPT, the two base detectors, the meta-learner that fuses them, and the purifier, is trained under supervision. Unsupervised learning uses no attack labels. It models the normal behaviour of a network and flags departures from that baseline as anomalies, which makes it attractive for catching zero-day attacks that a supervised model never saw during training. A canonical example in this domain is Kitsune, an ensemble of autoencoders that scores each packet by how poorly it reconstructs under a model of benign traffic [17]; it is also one of the independent detectors DEPT is evaluated in front of in Chapter 5. Semi-supervised learning sits between the two, exploiting a small labelled set together with abundant unlabelled data, and reinforcement learning replaces the fixed dataset with an agent that learns by interacting with an environment. DEPT uses neither semi-supervised nor reinforcement learning, although the latter is the natural framework for the continuous-adaptation extension discussed in Chapter 6. Self-supervised learning is a variant of unsupervised learning, deriving its training signal from the data itself rather than from annotations: TANTGen [1], the generator of the adversarial traffic used in this work, is pre-trained to reconstruct masked packet sizes and inter-arrival times on unlabelled traffic, and only its surrogate detector head is trained under supervision [1].

2.1.2 Feature Engineering and Representation

The performance of a learned detector depends directly on the representation it consumes. Feature engineering — in this case, the process of turning raw packets into numerical vectors — places an upper bound on what any model can learn [21, 22]. Before discussing representations, one term should be fixed: throughout this document, a flow is the sequence of packets that share the same five-tuple (source and destination Internet Protocol (IP) addresses, source and destination ports, and transport



(a) Supervised learning: labelled flows train a model that maps a flow to a label.

(b) Unsupervised learning: with no labels, the model learns the structure of the traffic and flags what departs from it.

Figure 2.1: The two supervision regimes most relevant to DEPT, on network flows. All of DEPT's components are supervised.

protocol), ordered by arrival time.

Two axes organize the design space of traffic representations. The first is granularity. Static, packet-level features are read from individual headers (addresses, ports, flags); they are cheap enough to extract at line rate, but they carry no temporal context. Dynamic, flow-level features aggregate a sequence of packets into statistics such as mean packet size, inter-arrival times, or flow duration. These capture behaviour, but at the cost of latency, since the system must wait for the flow to progress [14]. The second axis concerns the space in which a perturbation is expressed: the traffic-space of actual packets on the wire versus the feature-space of derived vectors. This distinction becomes central once adversaries are considered (Section 2.2.3). DEPT works at both granularities, and deliberately keeps its features close to the traffic-space so that its outputs can be turned back into real packets.

Rather than hand-designing features, DEPT represents traffic with **nPrint**, a bit-level encoding of packet headers [23]. Hand-crafted feature sets usually assume in advance which fields matter, and they are tied to particular protocols: a tool that computes flow statistics for Transmission Control Protocol (TCP) says nothing useful about an Internet Control Message Protocol (ICMP) flood. nPrint instead maps each bit of a packet's headers to one feature in a fixed-width vector, using a tri-state encoding: a value of 1 or 0 records a header bit that is set or unset, and -1 marks a field that belongs to a protocol layer absent from that packet. The representation is uniform across protocols, requires no manual feature design, and can be consumed directly by models that learn their own features. Bit-level encodings of this kind have proven effective for traffic analysis in settings where hand-crafted features generalize poorly [23–25]. Chapter 4 details the exact encoding DEPT derives from it.

A bit-level header vector does, however, inevitably include fields that identify the particular hosts and ports of a capture. If these are left intact, a model can reach high benchmark accuracy simply by memorizing which hosts appear in the attack traces, instead of learning the attack behaviour itself; this

shortcut inflates test scores but does not survive deployment on a different network [14, 26], which is why DEPT masks those fields (Chapter 4).

2.1.3 Model Architectures Relevant to DEPT

DEPT combines three architecture families, chosen for complementary properties rather than for raw accuracy alone.

Random forests. An RF is an ensemble of decision trees trained by bagging (Section 2.3.1): each tree recursively partitions the feature space with axis-aligned threshold tests, and the forest averages their votes [27, 28]. Forests are a strong, low-cost fit for tabular and statistical features and need no feature scaling [29], which is why DEPT applies one to flow-level statistics. Their decisive property for this work, however, is structural: a forest's decision function is constant over axis-aligned regions and therefore non-differentiable. It exposes no informative gradient that an attacker can follow, a property whose relevance for adversarial robustness is developed in Section 2.3.2.

One-dimensional and temporal convolutional networks. A Convolutional Neural Network (CNN) slides learnable kernels over its input to detect local patterns; in one dimension it processes sequences, extracting recurring motifs from a packet stream such as header patterns or sizing signatures [24, 25] (Figure 2.2). A TCN specializes this design for long sequences through two devices [30]. Dilated convolutions insert a growing stride between kernel taps, so the receptive field expands with depth without a corresponding blow-up in parameters [31]. Causal convolutions pad the input so that the output at position t depends only on positions $\leq t$, never on future packets. Causality is what makes the model valid for streaming inference: it can emit a verdict after each packet arrives.

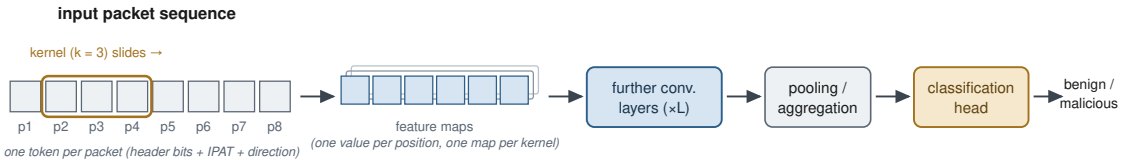


Figure 2.2: A one-dimensional convolutional pipeline over a packet sequence: kernels slide along the sequence and produce per-position feature maps, which further layers, pooling, and a classification head turn into a verdict. DEPT's temporal convolutional network specializes this design with dilated, causal convolutions.

Transformers. The self-attention mechanism lets every position in a sequence attend to every other position, weighting them by a learned measure of relevance [32]. Each input position is projected into a query, a key, and a value vector; collecting these as the rows of matrices Q, K, and V, attention computes

$$\mathrm{Attention}(Q, K, V) \;=\; \mathrm{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V, \tag{2.1}$$

where d_k is the key dimension, so the output at each position is a weighted combination of the values

of the positions it attends to. This captures long-range dependencies that a fixed convolution window cannot. Two arrangements are relevant here. Encoder-only models attend bidirectionally over the entire sequence, which suits classification and representation tasks, including learned traffic analysis [33, 34]. Decoder-only models apply a causal mask so that each position attends only to the past (Figure 2.3); this is the arrangement behind autoregressive generation, but the mask is needed by any task whose output at a position may depend only on that position's past, generative or not. Causality also enables key–value caching: the keys and values of past positions in Equation (2.1) do not change as new tokens arrive, so they can be stored and reused instead of reprocessing the whole prefix on every packet.

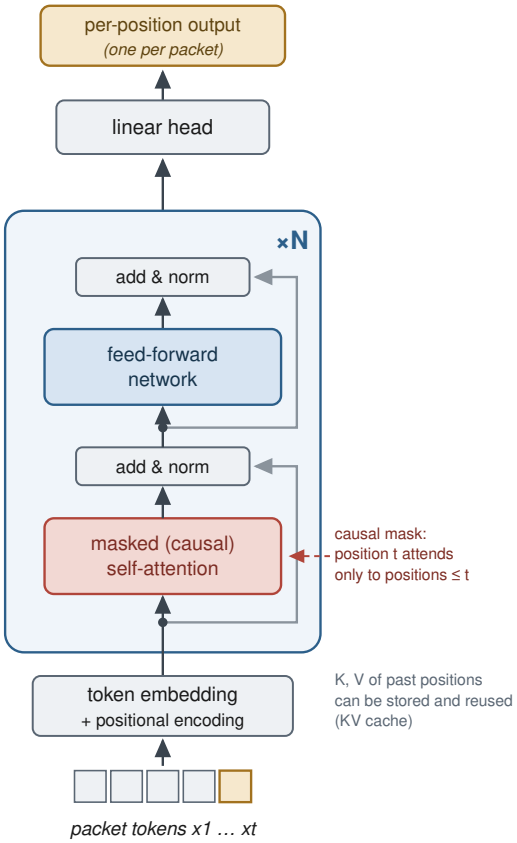


Figure 2.3: A decoder-only Transformer block over a stream of packet tokens: masked self-attention, then a feed-forward sublayer, with residual connections and layer normalization, stacked N times. The causal mask restricts each position to its past, which is what makes the design valid for streaming inference. Both of DEPT's Transformer components use this block.

2.2 Adversarial Machine Learning

The same flexibility that lets a learned model generalize also exposes it to manipulation. An adversarial example is an input perturbed by a small, deliberate amount so that the model misclassifies it, often while remaining nearly indistinguishable from a clean input [7, 8]. For a NIDS, adversarial examples are the

mechanism of evasion: an attacker reshapes malicious traffic just enough for it to be labelled benign [9, 35]. Adversarial Machine Learning (AML) studies both how such manipulations are constructed and how models can be hardened against them.

2.2.1 Threat Taxonomy

Attacks are most commonly classified along two axes: what the adversary knows, and what the adversary wants.

Along the knowledge axis, a white-box attacker has full access to the model, including its architecture and weights, and can compute gradients of the loss with respect to the input to craft precise perturbations. A gray-box attacker knows only part of the pipeline, such as the feature set. A black-box attacker can only query the system and observe its outputs. The black-box setting is the realistic one for a deployed NIDS, which an attacker probes as an oracle, and such attacks typically rely on transferability: an adversarial example crafted against a substitute model often fools the target as well [13]. Along the goal axis, poisoning corrupts the model during training, whereas evasion manipulates inputs at inference time against an already-trained model. DEPT is designed for the black-box evasion threat, which is the case that dominates real deployments (Chapter 4).

Network intrusion detection differs from the image classification setting, where adversarial examples were first studied, by a constraint with no equivalent in the pixel domain: a perturbed feature vector must correspond to a packet or flow that is actually valid on the wire. Recovering a realizable packet from a desired feature vector is the **Inverse Feature Mapping (IFM)** problem, and it is frequently constrained or computationally intractable [11, 12, 36]; Section 2.2.3 returns to it. In particular, the gradient-based methods that dominate the image setting, such as the Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD) [8, 37], produce feature perturbations that need not correspond to any sendable packet, which limits their direct use in this domain.

2.2.2 Defense Paradigms

Defenses against evasion fall into a few broad families, with different strengths for traffic data. Adversarial training augments the training set with adversarial examples so that the model learns to classify them correctly [37]. It is effective against the perturbations it is trained on, but it is costly, it can reduce accuracy on clean traffic, and in networking it is only meaningful if the adversarial examples are themselves realizable rather than impossible feature vectors. DEPT's detectors are likewise trained on realizable adversarial traffic, although with a different goal: rather than learning to keep classifying a perturbed attack as an attack, they learn to recognize that a flow has been manipulated at all (Section 3.2).

Input transformation, or purification, is the family DEPT builds its purifier on. Instead of hardening the classifier, it transforms each input before classification in order to remove the perturbation, projecting the sample back onto the manifold of normal traffic, as Figure 2.4 illustrates. The idea originates with Defense-GAN, which projects each input onto the manifold captured by a generative adversarial network [16], and it has since moved to diffusion models [38] and to the network domain [2, 39]. DEPT's purifier pursues the same goal without being a generative model itself: it is a causal Transformer denoiser, trained on matched adversarial-to-clean pairs rather than on a model of the benign distribution it could sample from (Chapter 4).

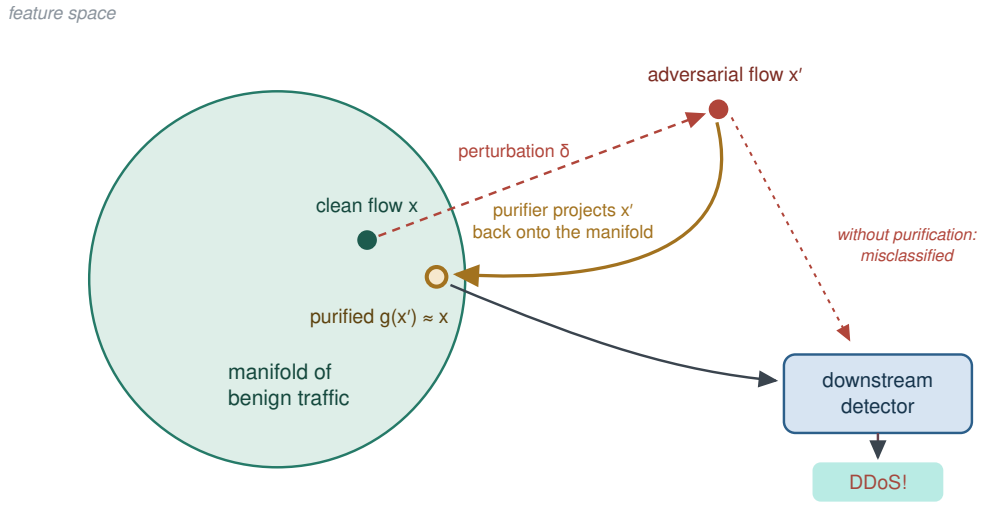


Figure 2.4: Purification as projection onto the benign manifold. A perturbation δ moves a clean flow x off the manifold, and the detector misclassifies the result x'. The purifier g projects x' back, so the detector receives an input close to x. The idea originates with Defense-GAN [16].

DEPT adopts neither of the two remaining families. Defensive distillation smooths the decision surface so that useful gradients become harder to find [40]. Certified robustness guarantees that no perturbation within a bounded region changes a prediction, through randomized smoothing [41, 42] or Lipschitz and edit-distance bounds [43]; such guarantees are hard to scale to the high-dimensional, discrete structure of traffic, and they secure a formal radius rather than the traffic-space validity that is central to this work.

2.2.3 Realizability and Protocol Constraints

The gap between a valid feature vector and a valid packet is a central difficulty of network adversarial machine learning. A perturbation can move a feature vector anywhere in $\mathbb{R}^n$, but a packet must satisfy hard structural constraints: checksums must match the bytes they cover, header-length fields must be equal to the actual lengths, control flags must be mutually consistent, and sequence numbers must be ordered. A perturbation that ignores these constraints is unrealizable: it describes no packet that

can be sent, so evaluating a detector against it overestimates the real threat [11, 12]. Recent attack research enforces realizability explicitly, whether by using an SMT solver to find packets that satisfy the protocol constraints, as in PANTS [44], or by generating traffic that is protocol-compliant by design, as in GPMT [36] and TANTGen [1].

The constraint applies to both sides, which is what makes it a design concern for DEPT rather than a property of attacks alone. A defense that edits traffic, as a purifier does, must itself re-emit valid packets; otherwise its output is unusable, or is trivially detected as malformed. The step from feature-space back to traffic-space, which we call realization, is therefore treated as a first-class part of the design. The nPrint representation helps here: because it is a bit-level image of the packet headers, a purified nPrint vector can be decoded back into a concrete packet and checked for on-the-wire validity, which DEPT does with packet-level tools (Chapter 4).

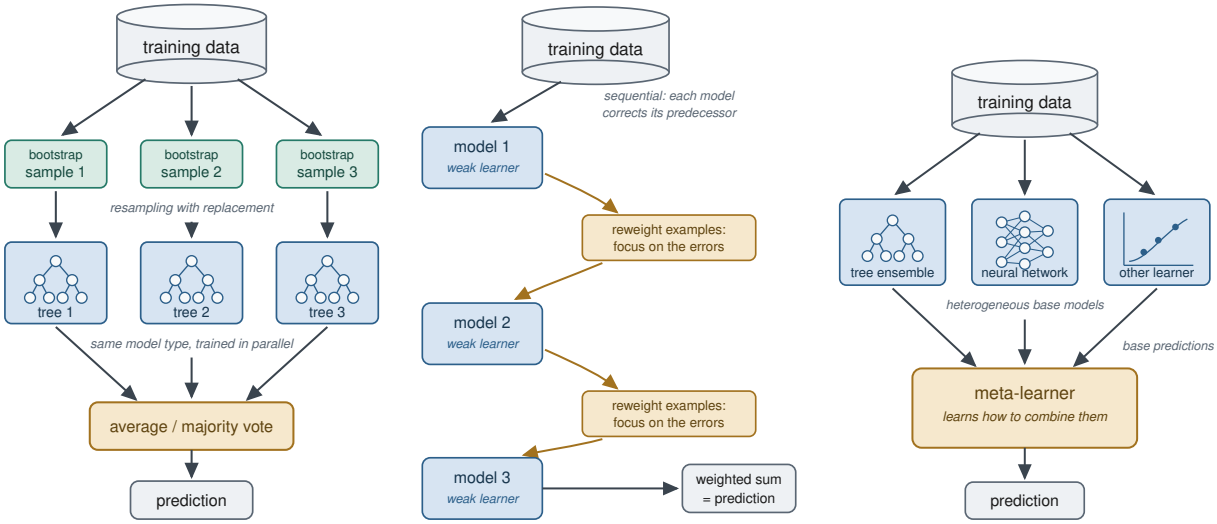
2.3 Ensemble Learning for Robustness

An ensemble combines several base models into a single predictor, on the premise that a collective decision is more accurate and more robust than that of any single learner [29, 45]. Beyond accuracy, ensembles are of particular interest for security, because diversity among the base models can be turned directly against adversarial transferability.

2.3.1 Bagging, Boosting, Stacking

Three combination strategies dominate (Figure 2.5). Bagging trains homogeneous models in parallel on bootstrap resamples of the data and averages their predictions, which reduces variance; the random forest is bagging applied to decision trees [27, 28]. Boosting trains models sequentially, each one focusing on the examples its predecessors got wrong, which reduces bias [46]. Stacking trains heterogeneous base learners and then trains a separate meta-learner to combine their outputs [15]. Stacking is most effective when the base learners are diverse and make different kinds of errors, so that their combination covers more of the input space than any of them alone.

Stacking is also the natural fit when the base learners are heterogeneous not only in architecture but in operation, for example when one emits a verdict on every packet and another only at sparse points of a flow. A fixed combination rule, such as averaging or majority voting, cannot express when each learner should be trusted. A learned meta-learner can: by assigning input and position-dependent weights to each base output, it acts as a dynamic gating mechanism rather than a static average [47, 48].



(a) Bagging: parallel homogeneous models on resampled data.

(b) Boosting: sequential models, each correcting the last.

(c) Stacking: heterogeneous models fused by a meta-learner.

Figure 2.5: The three classical ensemble strategies. DEPT's detector is a stacking ensemble, chosen because its base learners are architecturally heterogeneous and emit predictions at different rates.

2.3.2 Gradient Masking and Transferability

DEPT is itself a learned detector, so a sophisticated attacker may also try to evade DEPT directly. Such an attacker typically knows neither which models DEPT uses nor their parameters, the black-box setting of Section 2.2.1. Transferability is what makes black-box evasion practical: an attacker who cannot see the target model builds a differentiable substitute, crafts a perturbation against it with a gradient method such as PGD or FGSM, and relies on that perturbation carrying over to the target [13]. Heterogeneous stacking is a way of increasing DEPT's own robustness against this mechanism. A differentiable member, such as the TCN, exposes a gradient that an attacker can follow. An RF does not: its decision function is constant over axis-aligned regions, so there is no gradient direction to exploit. When a differentiable and a non-differentiable model are combined, a perturbation optimized against the differentiable member does not necessarily move the forest's discrete verdict, so it transfers poorly across the ensemble and raises the attacker's cost [48–51].

This argument must be stated carefully, because the absence of a usable gradient is a form of gradient masking, and gradient masking is not robustness in itself. A determined white-box adversary can estimate gradients numerically or resort to gradient-free search, and masked gradients can produce a false sense of security when mistaken for genuine robustness [52]. The argument holds for DEPT precisely because the attackers it is designed against have no white-box access to DEPT as a whole: the value of a non-differentiable member is not an absolute barrier, but a measurable reduction in the trans-

ferability from whatever substitute the attacker is able to build without direct gradient access. Robustness is therefore treated as a property to be evaluated empirically, not as a guarantee to be assumed, which is what Chapter 5 does.

2.4 Summary

The concepts in this chapter compose into the three pillars of DEPT. Representation: a bit-level nPrint encoding stripped of identity fields, paired with prefix statistics measured on the same packets, gives heterogeneous learners complementary behaviour-focused and realizable views of traffic (Section 2.1.2). Purification: an input-transformation defense, built as a causal Transformer, projects flagged traffic back onto the benign manifold while respecting the protocol constraints that keep its output realizable (Sections 2.2.2 and 2.2.3). Ensembling: heterogeneous stacking, fused by an attention meta-learner, combines a differentiable network with a non-differentiable forest to resist transferable gradient attacks under a black-box threat model (Section 2.3).

3

Related Work

Contents

3.1 Adversarial Attacks against ML-based NIDS 17
3.2 Training-time Defenses 19
3.3 Inference-time Defenses 20
3.4 Positioning of DEPT and Gap Analysis 22

The tension between learning-based detection and adversarial evasion has produced a large body of research on both sides. Chapter 2 introduced the concepts this work builds on; this chapter surveys the concrete systems, and reads each of them against the questions DEPT has to answer. The survey is selective rather than exhaustive: a work is emphasized when it shapes a design decision of DEPT, or when some part of DEPT departs from it.

3.1 Adversarial Attacks against ML-based NIDS

A defense is only meaningful relative to the attacks it expects, so the survey starts on the offensive side. Following the distinction of Section 2.1.2, attacks are divided by the space in which they operate: the feature-space of derived vectors, or the traffic-space of actual packets.

A – Feature-space attacks. The first wave of attacks against learned traffic detectors used the gradient-based methods of the image domain, FGSM and PGD among them [8, 37], onto NIDS feature vectors. Hashemi et al. [12] were among the first to evaluate state-of-the-art network detectors under such manipulations, and Tiki-Taka [10] made the threat concrete at scale: attacking deep learning-based intrusion detectors with a handful of black-box methods, it reached evasion rates of up to 35.7% against models that looked nearly perfect on clean benchmarks. Two lessons carry over to this work. First, a detector built from a single differentiable architecture inherits a smooth loss surface that gradient-guided search can exploit, directly or through a substitute model [13]; notably, two of Tiki-Taka's own defenses, model voting and ensemble adversarial training, already point towards the ensemble route DEPT takes. Second, most of these attacks perturb feature vectors without asking whether any real packet sequence maps to the perturbed vector, which is exactly the realizability constraint of Section 2.2.3 [11, 14]: their reported evasion rates are therefore upper bounds obtained in a space an external attacker does not control.

B – Traffic-space attacks. The recent attack literature has therefore moved into the traffic-space, enforcing realizability either by verification or by construction. **GPMT** [36] generates practical malicious traffic with a GAN under little prior knowledge of the target, constraining its perturbations so that the traffic keeps its malicious function. **PANTS** [44], a white-box tool, couples adversarial search with an Satisfiability Modulo Theories (SMT) solver that turns each adversarial input into a nearby packet sequence respecting the threat model and protocol semantics. **DeepRed** [53] embeds a GAN-based generator in a command-and-control framework and shows that realizable, protocol-abiding attacks degrade the detection rate of ML-based NIDS by up to 20%, which settles that the threat survives the realizability constraint. Most recently, **NetDiffuser** [54] brought diffusion models to the attack side, injecting semantically consistent perturbations into flow features, although still under a white-box assumption. The trend across these works is clear: the attack side has converged on generative sequence models operating under explicit realizability constraints, and it is to this class that the attacker of this thesis belongs.

3.1.1 TANTGen, the Attacker Used in This Work

TANTGen [1] is a Transformer-based adversarial network traffic generator developed in a sibling thesis, and it gets a closer treatment than the works above because it is the concrete adversary that DEPT is trained against and evaluated on. Where GPMT and DeepRed rely on GANs and PANTS on solver-checked perturbations, TANTGen uses self-attention to model the sequential dependencies between the packets of a flow, and generates its evasions in the traffic-space directly, so that protocol compliance holds by construction rather than only by after-the-fact verification. It operates in the black-box setting: it needs no access to the target detector's architecture, weights, or gradients, only to a surrogate NIDS of

its own, trained on labelled traffic, whose gradients guide the generator. Its evasion was demonstrated against the detectors its generation was calibrated on: Kitsune [17], an anomaly-based detector, for the Kitsune-native attacks, and its own ML-NIDS flow classifiers for the CIC attacks. The manipulation itself is confined to two quantities of live attack traffic, packet timing and packet size, exposed as three perturbation modes (`iat`, `size`, `both`) that structure the corpus of Section 4.3.

These properties, and not TANTGen itself, have three consequences that run through DEPT's design. First, since the adversarial traffic is protocol-compliant, a defense gains nothing from checking protocol validity; whatever signal exists must be found in the traffic's behaviour, its timing and sizing structure, which is what DEPT's learned detectors consume. Second, because the perturbation touches only timing and size, the detectable footprint can be thin, especially in the timing-only mode; Chapter 5 quantifies this. Third, the black-box setting makes transferability the operative attack channel, which is the scenario the heterogeneity argument of Section 2.3.2 is aimed at. The threat model of Section 4.1 formalizes these assumptions.

3.2 Training-time Defenses

Training-time defenses harden a model before it is ever deployed, by sanitizing its training data or by shaping its decision surface. Four lines of work are relevant here.

A – Data sanitization and poisoning defenses. The earliest defensive line protects the integrity of the training set itself. **Antidote** [55] replaces the principal-component subspace of an anomaly detector with robust statistics, so that poisoned flows that deviate from the benign subspace are rejected instead of taken in; its acknowledged difficulty is telling adversarial poisoning apart from the benign drift that network traffic produces naturally. A complementary result concerns bagging: Biggio et al. [50] showed empirically, and Chen et al. [51] later certified, that training each ensemble member on a random subset of the data dilutes the influence of any individual poisoned sample across the collective decision. DEPT excludes poisoning from its threat model (Section 4.1), so these defenses are not competitors; they matter here because they establish, from the training side, the same principle DEPT exploits at inference time, that ensemble diversity is a robustness mechanism in itself and not only an accuracy one.

B – Federated and backdoor defenses. As NIDS deployments move towards Federated Learning (FL) for privacy, a parallel line defends the training pipeline itself, detecting backdoors in the model updates clients contribute, in one case by having clients validate candidates inside Trusted Execution Environments (TEEs) [56–58], and unlearning them from weights already compromised [59–61]. These too defend a different adversary than DEPT's, and are complementary rather than competing.

C – Ensemble-based hardening. A third line builds robustness into the model itself by combining several learners; DEPT's detector is also an ensemble, but one used at inference time to recognize the manipulation. Beyond Tiki-Taka's voting ensembles [10], **TRS** [49] showed that low transferability among ensemble members can be promoted directly, by regularizing for gradient diversity and model smoothness during training, and **Stewart** [48] demonstrated, in the domain of predictive maintenance, that a stacking ensemble with a learned meta-model withstands white-box attacks substantially better than its individual members. **Mateen** [62] takes the ensemble idea in the direction of distribution shift, maintaining an adaptive committee of autoencoders for network anomaly detection, and broader benchmarking frameworks confirm the accuracy benefits of ensembling for intrusion detection in general [45]. What nearly all of these have in common is that their members are drawn from the same differentiable family, so a gradient exists for every member and the ensemble's protection rests on decorrelating those gradients. Heterogeneity across families, mixing a non-differentiable forest with a differentiable network so that part of the ensemble offers no gradient at all, is far less explored, and it is the specific form of diversity DEPT's detector is built on (Section 4.4).

D – Realizable adversarial training. Adversarial training [37] hardens a model by adding adversarial examples into its training set, but in the network domain it is only as good as the examples themselves: a model trained against impossible feature perturbations is hardened against a threat that cannot occur, the IFM problem of Section 2.2.1 seen from the defender's side. The works of Section 3.1 close this gap by supplying realizable adversarial data, and PANTS in particular demonstrates that inserting its realizable examples into the training loop yields robustness that persists against attacks not seen during training [44], while DeepRed argues the same necessity from the red-team side [53]. DEPT uses the same kind of data, but not to retrain the protected NIDS: its own detector and purifier are trained on realizable, black-box adversarial traffic generated by TANTGen, not on synthetic feature noise (Section 4.3). Adversarially trained classifiers aim to keep classifying perturbed attacks as attacks; DEPT aims to recognize the manipulation itself and then to undo it, so both of its stages belong to the inference-time family surveyed next.

3.3 Inference-time Defenses

Inference-time defenses act at deployment, on the inputs a model receives. Three groups are relevant: certifying predictions against bounded perturbations, purifying inputs before classification, and detecting adversarial inputs by auxiliary signals.

A – Certified robustness. Certification promises that no perturbation within a bounded region can flip a prediction. **CertTA** [42] made randomized smoothing practical for traffic analysis by drawing its smoothing noise jointly over the dimensions an attacker actually perturbs, packet timing, sizing, and insertion, and derives robustness regions that are meaningful for traffic models. In text classification, **LipsLev** [43] trains 1-Lipschitz classifiers whose certificates under a bounded Levenshtein distance cost a single forward pass. Both act in the two phases: the model is fine-tuned (CertTA) or trained (LipsLev) for the certificate, which is then computed for each prediction. The guarantee is valuable but bounded by construction: a certificate covers a radius, and a manipulation of timing or size is not constrained to stay inside one, since a flow's timing can be stretched by legitimately large amounts while remaining perfectly realizable.

B – Generative purification. The purification family goes back to **Defense-GAN** [16], which projects each input onto the manifold captured by a generative model before classification. Its modern descendants are mostly diffusion-based and mostly proven in the image and text domains [38, 63, 64], where "causal" denotes factor attribution rather than the temporal masking DEPT uses. In the network domain the idea has been instantiated more recently: **AdvPurRec** [39] reconstructs traffic features with a diffusion model before they reach the NIDS, and Merzouk et al. [2] map the trade-off that governs such purifiers, where more diffusion noise removes more of the perturbation but degrades clean accuracy.

DEPT's reconstructor (Section 4.5) shares this family's goal but departs from it on three axes. The first is architectural: a diffusion purifier needs the whole input before it can denoise it, and its cost grows with the number of denoising steps, whereas DEPT uses a causal Transformer that corrects each packet from the packets before it, in a single pass, and defaults to the identity (no-op) through its residual head. The trade is explicit: DEPT gives up the broad distribution coverage of a diffusion process in exchange for a model that fits streaming deployment. The second axis is supervision: purifiers in the Defense-GAN tradition learn from clean data alone and purify by projection, while DEPT exploits the matched adversarial-to-clean flow pairs its corpus provides, a stronger form of supervision that is normally unavailable, but that a paired generator such as TANTGen provides; Section 5.7.2 compares the two directly, with the purifier of Merzouk et al. as the baseline. The third axis is the one this thesis considers decisive: every network purifier surveyed above stops at the feature vector, and none of them re-emits traffic. A purified vector that corresponds to no sendable packet sequence has simply moved the realizability problem from the attacker to the defender. DEPT treats the step back into the traffic-space, realization followed by dissector-level validation, as part of the defense itself (Section 4.5.3).

C – Constraint-based and auxiliary detection. A final group detects adversarial inputs without a generative process. **NIDS-CBAD** [65] flags violations of inter-feature domain constraints, for instance a payload size inconsistent with a packet count, on the observation that naive feature-space attacks

often break such invariants. **AMDS** [66] fuses ensemble disagreement, predictive uncertainty, and distributional anomaly scores into an adversarial-input detector with attack-adaptive weights. DEPT adopts neither as its detection principle, for a reason that follows directly from Section 3.1.1: its adversary emits protocol-compliant traffic by construction, so constraint checks are blind to it, and the constraint family defends against exactly the naive attacks that the realizability literature has already moved past. The disagreement signal of AMDS is closer in spirit to this work: DEPT's meta-learner can in fact be read as a learned, per-packet version of it, fusing the opinions of two deliberately different base learners (Section 4.4.3).

3.4 Positioning of DEPT and Gap Analysis

Figure 3.1 arranges the surveyed defenses by what they do and by the space in which they work, and Table 3.1 lists the same works with their techniques. Both also record the phase at which each defense acts. A defense acts at training time when it changes the protected model, while or after it is trained, and at inference time when it works on each input or prediction and leaves the model as it is; certification typically uses both phases (Section 3.3).

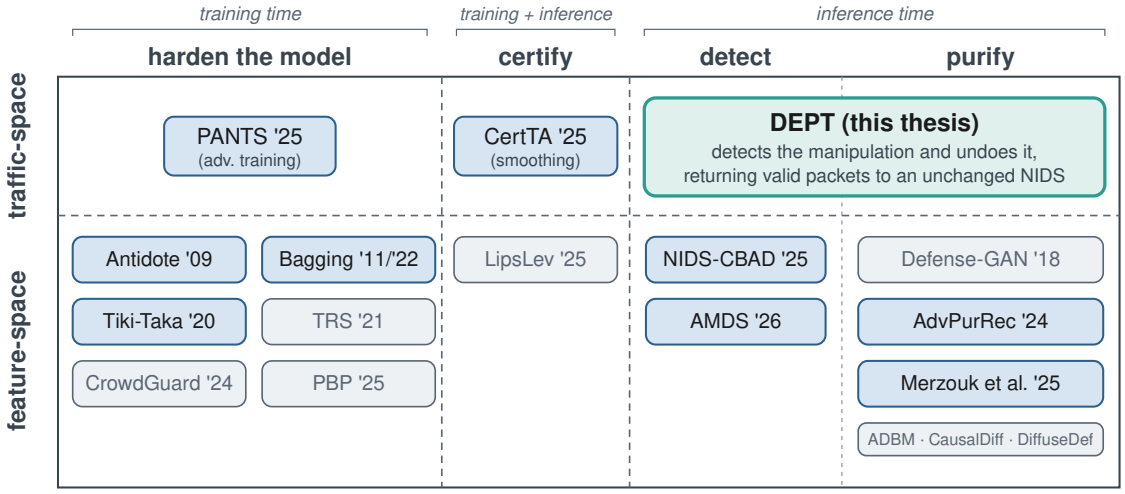


Figure 3.1: The surveyed defenses arranged by what they do (columns, labelled with the phase at which they act) and by the space they work in (rows). Grey boxes are works from outside the network domain. In the traffic-space row, no surveyed defense detects or purifies manipulated traffic; DEPT does both, at inference time and without changing the protected NIDS.

The figure shows where the surveyed work stops. In the feature-space row, every strategy has examples: models are hardened, predictions certified, and manipulated vectors detected or purified. In the traffic-space row, a defense either hardens the model in advance, as PANTS does by adversarial training, or certifies a prediction within a bounded region, as CertTA does. None of them takes a flow as it arrives, decides whether it was manipulated and gives the NIDS the flow back with the manipulation

Table 3.1: Comparison of the surveyed adversarial defenses and of DEPT. Phase: $\mathcal{T}$, the defense changes the protected model, while or after it is trained; $\mathcal{I}$, it acts on each input or prediction and leaves the model unchanged. Scope: Φ feature-space, Ψ traffic-space. †Outside the network domain, where perturbations act directly on the model's input vector.

System	Year	Strategy	Phase	Scope	Techniques
Antidote [55]	2009	Statistical sanitization	$\mathcal{T}$	Φ	Robust statistics
Bagging defense [50, 51]	2011	Poisoning defense	$\mathcal{T}$	Φ	Bagged ensemble
Tiki-Taka (defense) [10]	2020	Ensemble hardening	$\mathcal{T}$	Φ	Voting, adv. training
TRS† [49]	2021	Ensemble hardening	$\mathcal{T}$	Φ	Gradient diversity
CrowdGuard† [58]	2024	FL backdoor defense	$\mathcal{T}$	Φ	TEE, clustering
PBP† [61]	2025	Backdoor unlearning	$\mathcal{T}$	Φ	Neuron masking
PANTS (defense) [44]	2025	Adversarial training	$\mathcal{T}$	Ψ	SMT-constrained examples
CertTA [42]	2025	Certified robustness	$\mathcal{T}+\mathcal{I}$	Ψ	Multi-modal smoothing
LipsLev† [43]	2025	Certified robustness	$\mathcal{T}+\mathcal{I}$	Φ	Levenshtein certificates
Defense-GAN† [16]	2018	Purification	$\mathcal{I}$	Φ	GAN projection
AdvPurRec [39]	2024	Purification	$\mathcal{I}$	Φ	Diffusion
Merzouk et al. [2]	2025	Purification	$\mathcal{I}$	Φ	Diffusion
NIDS-CBAD [65]	2025	Adversarial detection	$\mathcal{I}$	Φ	Domain constraints
AMDS [66]	2026	Adversarial detection	$\mathcal{I}$	Φ	Ensemble signals
DEPT (this thesis)	2026	Detection + purification	$\mathcal{I}$	Ψ	Heterogeneous stacking, causal purifier, realization into packets

undone. The network purifiers of Section 3.3 do undo manipulations, but only on feature vectors, so their output serves only a detector that reads those same features.

4

Solution: DEPT Architecture

Contents

4.1 Threat Model 26
4.2 System Overview 27
4.3 Dataset Construction and Packet-to-Tensor Pipeline 29
4.4 Primary Detection: Heterogeneous Stacking Ensemble 32
4.5 Input Purification: Reconstructor 35
4.6 Implementation and Tools 37

This chapter presents DEPT (Defense via Ensemble and Purification Transformer), the system this thesis proposes and builds. DEPT composes the three pillars introduced in Chapter 2 into a single streaming defense: two traffic representations emitted at different rates, a heterogeneous detection ensemble fused by an attention meta-learner, and a causal purification Transformer whose output is realized back into valid packets. The chapter is organized as follows. Section 4.1 fixes the threat model, since later decisions are justified relative to it. Section 4.2 gives the end-to-end view of the architecture and its two operating phases. Section 4.3 describes how the training corpus is constructed and how raw captures become the tensors the models consume. Sections 4.4 and 4.5 then detail the two functional stages, detection and purification, and Section 4.6 closes with the implementation. Design decisions

and their rationale belong to this chapter; the measured behaviour of the resulting system is the subject of Chapter 5.

4.1 Threat Model

Three kinds of traffic are distinguished. Benign traffic is the normal traffic of a network, with no attack in it. Attack traffic belongs to a conventional attack, such as a flood or a scan, of the kind a NIDS is built to detect. Adversarial traffic is attack traffic that has been perturbed so that the NIDS no longer detects it; these perturbations are what DEPT defends against.

The scenario, illustrated in Figure 4.1, involves three parties. A protected NIDS guards a network and is assumed to handle benign and attack traffic correctly, but not adversarial traffic; this is the detector an adversary wants to get past. An attacker produces adversarial traffic by manipulating an ongoing attack, the evasion attack of Section 2.2. DEPT sits inline, immediately upstream of the protected NIDS: its job is to decide, packet by packet, whether a flow is adversarial, and to neutralize the manipulation before the protected system ever consumes the flow. Since DEPT only separates adversarial traffic from the rest, the label convention of Section 4.3 groups benign and attack traffic together as benign, in the sense of unmanipulated.

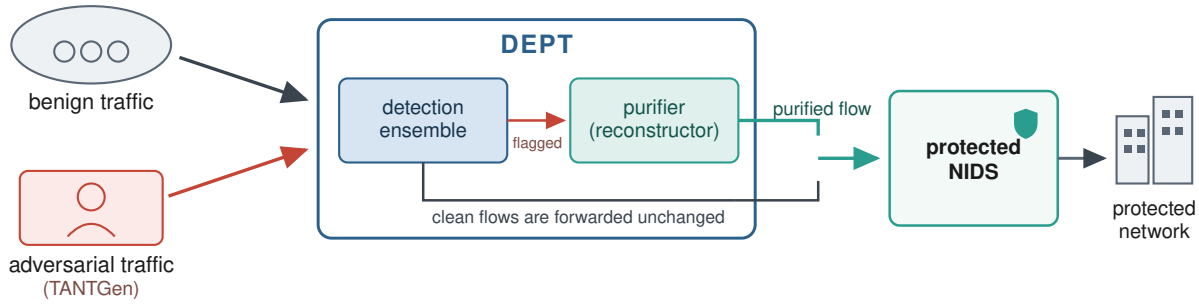


Figure 4.1: DEPT's deployment context. Benign and manipulated traffic (generated by TANTGen in this work) arrive interleaved. Flows judged clean are forwarded unchanged, and flagged flows pass through the purifier first, so the protected NIDS receives traffic whose manipulation has been undone.

The attacker is any adversary with the capabilities below; in this work it is TANTGen [1] (Section 3.1.1), which ties them to an attack that has been demonstrated rather than assumed. It works in the black-box setting of Section 2.2.1: it has no access to the architecture, parameters, or gradients of DEPT or of the protected NIDS, and can at most observe detection outcomes. It perturbs two quantities of live attack traffic, packet timing (inter-arrival times) and size (payload padding), alone or together (TANTGen's `iat`, `size`, and `both` modes). Its packets are protocol-compliant. Attacks that break protocol semantics, such as a wrong checksum or a negative inter-arrival time, are out of scope: they are not working evasions (Section 2.2.3), and stateless checks reject them before any learned model is involved.

TANTGen takes its gradients from its own surrogate NIDS, never from DEPT, so it cannot test the claim behind the ensemble, which is a claim about gradient paths. A second adversary, evaluated in Section 5.5, therefore has white-box access to the TCN (architecture, weights and gradients) and nothing for the Random Forest, which has no gradient to leak. Its perturbations are protocol-compliant and increase-only, since packets on the wire can be padded or delayed but not un-sent, and its budget is counted in bytes of padding and seconds of delay.

DEPT, in turn, does not use TANTGen's model, parameters, or surrogate NIDS, only its traffic: labelled flows train the detectors, and flows paired with their clean originals train the reconstructor (Section 4.5.2), so any generator with the capabilities above could replace TANTGen. It is not black-box all the way through, however, since realization writes back only the two quantities the attacker perturbs (Section 4.5.3). DEPT thus knows the attacker's feature set but not its model, the gray-box setting of Section 2.2.1 seen from the defender's side. Training and test flows come from the same generator, so DEPT is evaluated against the attacker it was trained on, and other generators are left to future work (Section 6.4). Only the second adversary's added perturbations are never seen in training.

Three non-goals complete the model. DEPT does not defend against poisoning: its training data comes from known sources and the training pipeline is trusted. No claim is made against an adversary with gradient access to every component, which for the Random Forest is impossible. And DEPT does not replace the protected NIDS: detecting the underlying attacks (a flood, a scan) stays the NIDS's job, and DEPT only answers whether a flow has been manipulated, which is what the label convention of Section 4.3 encodes.

4.2 System Overview

Figure 4.2 shows the complete architecture. Traffic enters through the Packet-to-Tensor pipeline, which derives two views of each flow (Section 4.3) at two rates: a dense stream of masked nPrint per-packet tokens and a sparse stream of statistics measured on the packets themselves. The two base learners consume one stream each. The TCN produces a verdict logit at every packet, from the very first one; the RF produces a logit at packet 2 and then every tenth packet, on the aggregated prefix. A per-packet fusion token joins the two prediction streams with a small amount of context, and the meta-learner, a compact causal Transformer, turns that token stream into a per-packet probability that the flow is adversarial. Flows that never cross the decision threshold are forwarded untouched. Flows that do are handed to the reconstructor, a causal purification Transformer that undoes the manipulation in feature space, and a realization step then writes that correction back onto the traffic space. Realization is the last stage of the inference path: what reaches the protected NIDS is the realized traffic, not the purified tensors.

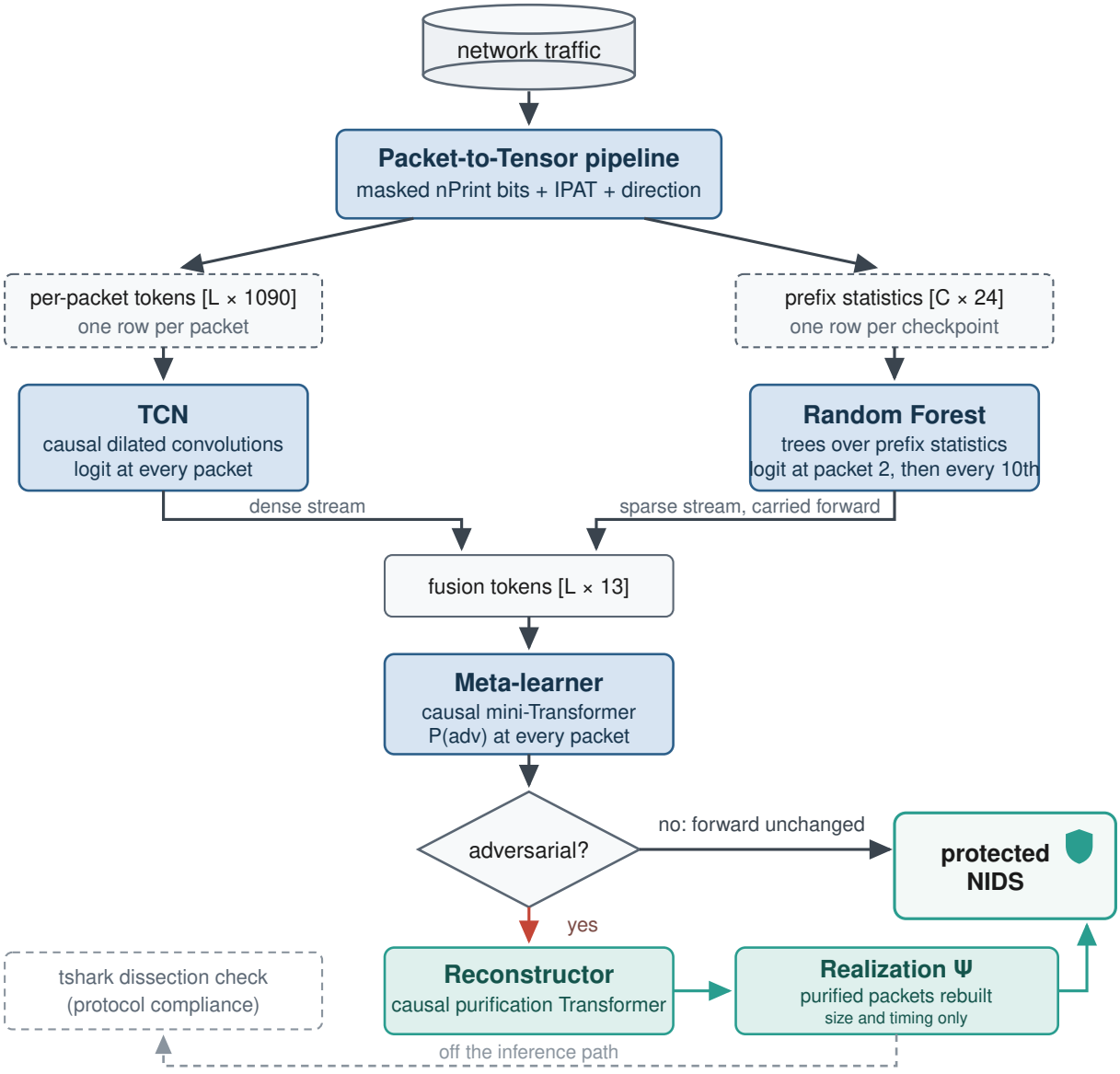


Figure 4.2: The DEPT architecture. The Packet-to-Tensor pipeline feeds two representations of the same flow to two architecturally different base learners; their prediction streams are joined into 13-dimensional per-packet fusion tokens and fused by a causal mini-Transformer into a streaming verdict. Only flagged flows pay the cost of purification.

The shape of the system follows from the objectives stated in Section 1.3. The dual representation answers O1: both learners see the same flow through a behaviour-focused, identity-free view, each at the granularity it suits. The ensemble answers O2: the two base learners differ in architecture (non-differentiable trees against a differentiable network), in input (aggregated statistics against raw sequences), and in timing (sparse against dense), which is the diversity that the transferability argument of Section 2.3 asks for. The reconstructor and the realization step together answer O3, and the per-packet verdicts, rather than end-of-flow ones, are what O4's evaluation measures.

DEPT operates in two phases. In the training phase, all components are fitted offline, in dependency order. The pipeline first tensorizes every source once. The two base learners are then trained independently on the training split. The meta-learner is trained next, on the validation split rather than on the training split, for the reason given in Section 4.4.3. The reconstructor is trained last, on matched pairs of adversarial and clean flows. In the deployment phase the flow of Figure 4.2 would run continuously: tensorize each arriving packet, run through both base learners, fuse, and decide. Purification is gated on the ensemble's flag, so the common case, unflagged clean traffic, never pays for it, and only the flows the detector marks adversarial are handed to the reconstructor and then realized back into packets before reaching the protected NIDS.

4.3 Dataset Construction and Packet-to-Tensor Pipeline

A – Traffic corpus. The corpus pairs ten traffic sources with adversarial counterparts generated by TANTGen. Five sources carry over from the datasets most used in the related literature: the Mirai botnet and SSDP flood captures of Kitsune [17], and the Distributed Denial of Service (DDoS), port scan, and FTP/SSH brute-force captures of CIC-IDS2017 [18]. Five more come from CIC-DDoS-2019: four reflection and amplification floods (NTP, DNS, SSDP, and LDAP) and a SYN flood [19]. For each source, TANTGen produced one adversarial variant per perturbation mode (`both`, `iat`, `size`), giving thirty adversarial source–mode combinations. Two construction decisions should be mentioned. The CIC-DDoS-2019 captures are attack-dominated from their first seconds and offer no clean time window to filter on, so each is truncated to its first million packets instead, the same scale at which the generator caps its own malicious input. And no source without a TANTGen counterpart was admitted into the corpus: benign flows from unpaired captures would inflate the benign class without contributing any adversarial signal to learn against.

B – Label convention. Every flow taken from the raw datasets is labelled benign, including their native attacks, and every TANTGen-generated flow is labelled adversarial. This follows directly from the threat model: detecting a flood or a scan is the protected NIDS's responsibility, while DEPT's task is to

recognize whether a flow has been manipulated. The labels therefore encode the question the system must answer, “did the generator touch this flow”. Throughout the rest of this document, benign is used in this sense of unmanipulated: the ten original sources are called benign even though they are drawn from attack captures. One consequence is that, on a source whose features TANTGen barely moves, the two classes are nearly indistinguishable for any detector; Section 5.4.2 shows this cost.

C – From packets to tensors. Each capture is processed in a single pass. Packets are grouped into flows by their canonical five-tuple and kept in timestamp order, and each packet is encoded into a 1090-dimensional token (Figure 4.3). Following nPrint (Section 2.1.2), DEPT encodes the IPv4, TCP, User Datagram Protocol (UDP), and ICMP headers, producing a 1088-bit vector ($480+480+64+64$), and appends two derived per-packet scalars, the z-scored Inter-Packet Arrival Time (IPAT) and a direction bit (the normalization statistics are computed at tensorization time and stored alongside the tensors, so that training and inference standardize identically). The per-flow token sequences are stored as one large ragged array with an offsets table, which keeps loading cheap and lets every downstream component share the same on-disk artifacts.

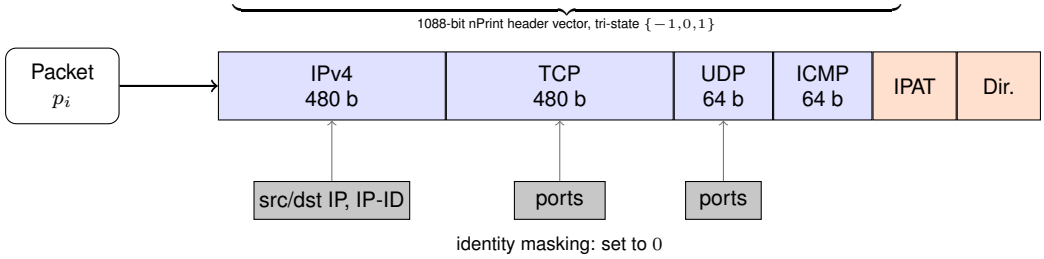


Figure 4.3: The per-packet representation used by DEPT: the four header blocks encoded bit-by-bit into 1088 tri-state cells following nPrint [23], the identity fields masked to zero, and the two derived scalars appended to give a 1090-dimensional final token.

The header bits include the fields that identify the particular hosts and ports of a capture: the source and destination IP addresses, the transport ports, and the IP identification field. These are the fields behind the memorization shortcut described in Section 2.1.2, so DEPT masks them, zeroing the 144 bits that encode them (Figure 4.3). Masking forces the detectors to rely on behavioural structure, such as packet sizing and timing, rather than on the identity of the hosts of one capture.

The same pass also produces the sparse representation for the RF, measured on the packets themselves rather than derived from the tokens (Table 4.1). At each checkpoint, six base features (inter-arrival time, packet size, payload length, payload entropy, TCP flags, and direction) are summarized by their mean, standard deviation, minimum, and maximum over the whole prefix seen so far, a 24-dimensional vector. None of the six encodes host or port identity, so this stream needs no masking of its own. These statistics are defined incrementally, by Welford’s online algorithm for means and variances together with

running extremes, so that a flow never needs to be re-buffered: the cost per packet is constant. Checkpoints fire at packet 2 and then every tenth packet. The early checkpoint is deliberate: some attacks generate extremely short flows. A port scan, for example, averages roughly two packets per flow, typically a single SYN and its RST, so without the checkpoint at packet 2 the forest would never issue a verdict for precisely the flows that a scan consists of.

Table 4.1: The two representations DEPT derives from the same packets.

Representation	Consumer	Emission rate	Dimension
Per-packet token	TCN	every packet	1090 (nPrint $+$ IPAT $+$ direction)
Prefix checkpoint	Random Forest	packet 2, then every $10th$	24 (mean/std/min/max of 6 features)

The two streams also differ in content and not only in rate: nPrint encodes headers only, so the payload entropy the forest keys on is structurally invisible to the TCN. Feeding one learner a dense per-packet sequence and the other a sparse, aggregated summary of a partly different view is also a source of the ensemble diversity that Section 2.3 argues to be valuable. Figure 4.4 lays the resulting prediction pattern out on the packet axis, including the carry-forward behaviour that the fusion stage of Section 4.4.3 builds on.

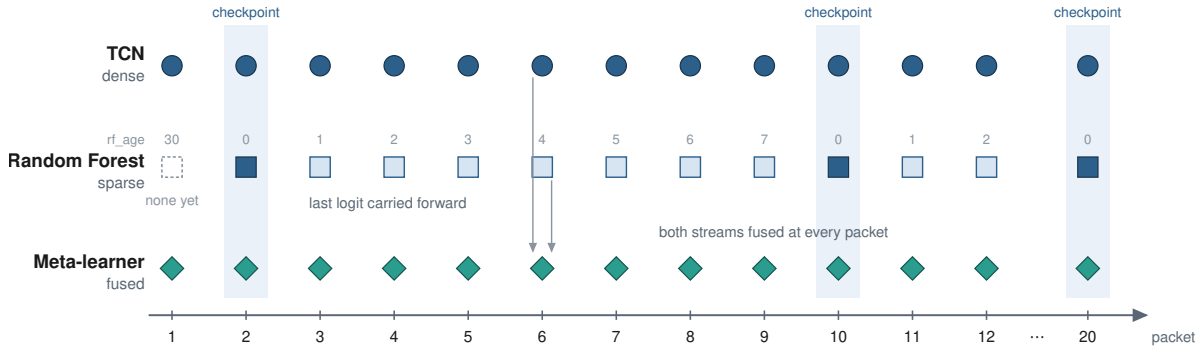


Figure 4.4: The dual prediction cadence over the first packets of a flow. The TCN emits a verdict at every packet, starting at the first. The Random Forest emits only at checkpoints (packet 2, then every tenth packet); between checkpoints its last logit is carried forward, and before packet 2 it has no verdict at all. The `rf_age` row shows the recency counter that the fusion token exposes to the meta-learner. The meta-learner fuses both streams into a fresh verdict at every packet.

D – Splits and class balance. All models share a single $70/15/15$ train/validation/test split, drawn with seed 42. The split is made at flow level: every row derived from a flow, be it a packet token, a checkpoint vector, or a reconstruction pair, lands on the same side, since splitting per row would leak near-duplicate prefixes of the same flow across sets. It is also stratified by source, so each split preserves the corpus's source mixture, and it is equally reused by the RF, the TCN, the meta-learner, and the reconstructor. This shared split is what later makes their comparison like-for-like, and it guarantees that the meta-learner, which consumes the base learners' outputs, is never evaluated on flows any

base learner trained on. Class imbalance is handled by the learners themselves, with balanced class weighting in both the forest and the neural losses.

4.4 Primary Detection: Heterogeneous Stacking Ensemble

The detection stage instantiates the stacking pattern of Section 2.3: two deliberately opposite base learners and a learned combiner. The two learners, an RF and a TCN, are heterogeneous in architecture and asynchronous in operation: the network emits a dense verdict on every packet, while the forest emits a sparse verdict only at its checkpoints, starting at packet 2. This is the setting stacking was designed for, and the reason DEPT fuses them with a learned meta-learner rather than a fixed rule such as averaging or majority voting (Section 2.3.1). The base learners are described first, the fusion afterwards.

4.4.1 Random Forest Base Learner

The RF consumes the 24-dimensional checkpoint vectors and answers, at every checkpoint, whether the prefix statistics look manipulated. The choice of a forest for this role is doubly motivated. Practically, forests are strong learners for small tabular inputs and require neither feature scaling nor much tuning (Section 2.1.3). Strategically, a forest's decision function is piecewise constant and non-differentiable, which makes it the gradient-free member that the transferability argument of Section 2.3.2 requires: the forest's verdicts give no smooth loss surface, even to a surrogate-based attacker.

The configuration was selected by grid search on the validation split, over $\{200, 400\}$ trees, maximum depths $\{\text{unbounded}, 20, 40\}$, and minimum leaf sizes $\{1, 2, 5\}$, with $\sqrt{d}$ feature sampling and the `balanced_subsample` class weighting fixed throughout. The selected model uses 200 trees of maximum depth 20 with at least 5 samples per leaf. The search favoured the most regularized corner of the grid: with only 24 input dimensions, deeper, unconstrained trees mostly memorize checkpoint idiosyncrasies, while a bounded depth and a non-trivial leaf size force better generalization across flows. At inference time the forest is queried at the pipeline's checkpoints, so its verdict stream is sparse, delayed by design, and at the cadence of Figure 4.4.

4.4.2 Temporal Convolutional Network Base Learner

The TCN is the dense half of the ensemble. It consumes the full per-packet token stream and emits a logit pair at every position, so a flow acquires its first verdict with its first packet. The network instantiates the architecture of Section 2.1.3: three residual temporal blocks of 128 channels each, with two convolutions per block, kernel size $k = 3$, dilations 1, 2, and 4, and dropout of 0.15, closed by a per-timestep linear head; in total about 0.8 million parameters. Causal padding keeps every output independent of

future packets, which is the property that makes the model legitimate for streaming use, and the dilation schema gives a receptive field of

$$\mathrm{RF} \;=\; 1 + 2\,(k-1)\sum_{i=0}^{2} 2^{\,i} \;=\; 1 + 2 \cdot 2 \cdot (1+2+4) \;=\; 29 \text{ packets}, \tag{4.1}$$

so a verdict at any packet integrates evidence from that packet and the 28 preceding it (Figure 4.5), enough context to cover several RF checkpoint intervals (of roughly 10) without inflating the parameter count.

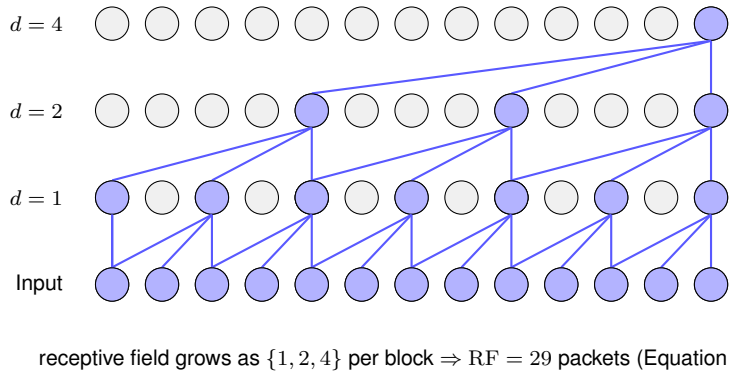


Figure 4.5: Dilated causal convolutions. Highlighted nodes trace the dependency cone of the rightmost output: dilation grows with depth $(1, 2, 4)$, so the receptive field widens while every output still depends only on the present and past. This is a schematic representation, with one convolution per block drawn; DEPT stacks two, giving the 29 packets of Equation (4.1).

Training minimizes per-packet cross-entropy with balanced class weights, averaged within each flow before averaging across flows, so that a ten-thousand-packet flood does not drown out two-packet scans in the gradient. Sequences are truncated at 200 packets when batching, so that a single long flow does not force the whole padded batch to its length; truncation therefore discards supervision beyond packet 200 rather than distorting it, which is consistent with the early-detection goal. Optimization uses AdamW with a cosine learning-rate schedule and early stopping on the validation split. The intended division of labour inside the ensemble is thus clear: the TCN contributes an early, dense, but noisier signal, the forest a delayed, sparse, more accurate one, and neither is expected to be sufficient alone.

4.4.3 Meta-Learner (the "Mini-Transformer")

The meta-learner reconciles the two prediction streams. The difficulty is not accuracy but asynchrony: at most packets the forest has no fresh opinion, and before packet 2 it has none at all, so a fixed rule such as averaging is not even well-defined at every position. DEPT's answer is a small decoder-only Transformer over per-packet fusion tokens, whose 13 dimensions are listed in Table 4.2. The token gives the model the two base logits, the information needed to interpret them (how stale the forest's logit is,

whether it was refreshed at this very packet), and a deliberately compact summary of the current packet.

Table 4.2: The 13-dimensional per-packet fusion token consumed by the meta-learner.

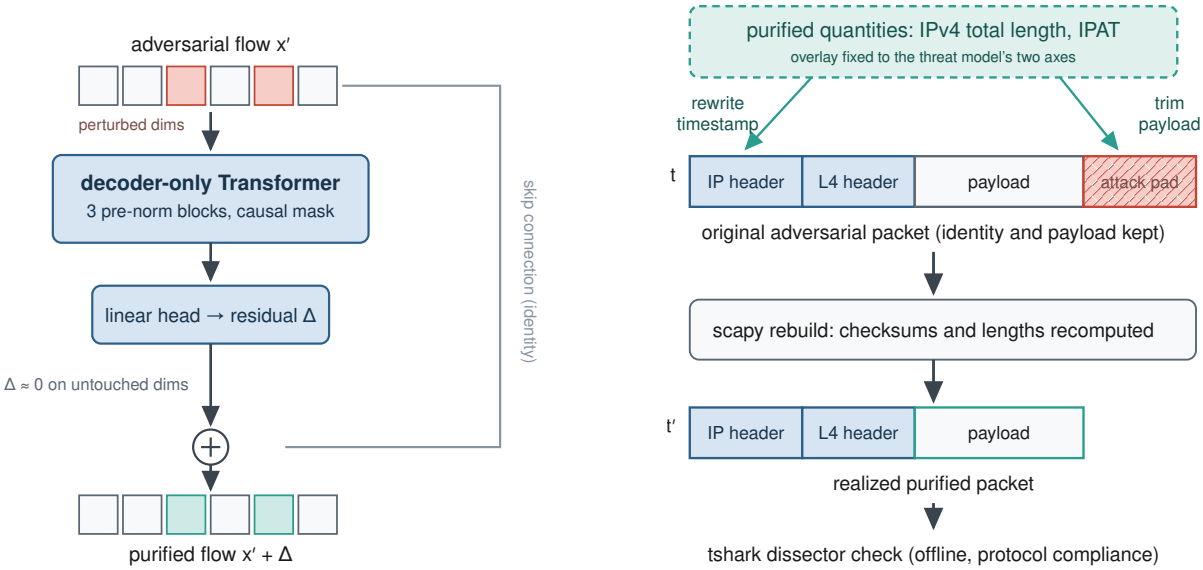
Slot	Feature	Meaning
0	RF logit	last checkpoint logit, carried forward; placeholder before packet 2
1	TCN logit	dense per-packet logit
2	`rf_age`	packets since the last RF refresh, clipped at 30; 30 before packet 2
3	`is_rf_checkpoint`	1 iff the RF logit was refreshed at this packet
4	IPAT	z-scored inter-packet arrival time
5	direction	direction bit of the current packet
6	packet length	IPv4 total length, normalized by 1500
7–12	TCP flags	URG, ACK, PSH, RST, SYN, FIN bits

Three design decisions define this component. First, the input is kept minimal on purpose: the meta-learner is a combiner of the base learners, not another feature extractor, and withholding the full 1088-bit representation forces it to combine the base learners instead of bypassing them; it also keeps the model at roughly 68k parameters (two causal blocks, four heads, $d_{\text{model}} = 64$), negligible next to either base. Second, the pre-checkpoint gap is filled by a learnable placeholder logit rather than an imputed constant. A zero would be read as a genuine "undecided" forest opinion; a trainable parameter lets the model learn its own prior for "the forest has not spoken yet". Third, attention is causally masked and positions are encoded with the standard sinusoidal scheme, so the fused verdict at packet t depends only on packets up to t, preserving the streaming property end to end.

The training discipline matters as much as the architecture. Fitting the meta-learner on the same data its base learners trained on would let it learn their overfit confidence rather than their generalizing behaviour [15], so the meta-learner trains on the validation split, where both base learners' predictions are out-of-fold, and that split is sub-divided (seed 43) into a meta-train and a meta-val part, the latter using early stopping on streaming Area Under the Receiver Operating Characteristic curve (AUROC) at packet 10. The loss is the same balanced per-packet cross-entropy as the TCN's, plus an auxiliary "beat the best base" term (weight 0.5) that penalizes, at each packet, any shortfall of the meta-learner's true-class confidence against the better of the two base learners. The intent is to make "never worse than the best base" an explicit training signal; what this loss does and does not achieve in practice is reported in Section 5.4.1. At deployment the meta-learner's per-packet probability is compared against a fixed threshold of 0.5; a flow is flagged the first time its score crosses it, which is also the event the earliest-detection metric of Chapter 5 uses.

4.5 Input Purification: Reconstructor

Detection alone would leave only a blunt response to a flagged flow, dropping it, which turns every false positive into a denial of service for a legitimate client. DEPT's second stage takes the purification route of Section 2.2.2 instead: undo the manipulation and let the repaired flow proceed. The reconstructor is a causal Transformer denoiser operating directly on the per-packet token sequence, trained to map an adversarial flow onto its clean counterpart (Figure 4.6). It is therefore not a generative purifier in the sense of Section 2.2.2: it models no distribution over benign traffic and samples nothing, but regresses the correction directly.



(a) Feature-space purification: a decoder-only Transformer predicts a per-dimension residual correction Δ, and the purified flow is $x' + \Delta$.

(b) Traffic-space realization: the purified length and timing are written back onto the original packets, and the result is checked with a dissector.

Figure 4.6: The purification stage. Purification happens in feature space (a), over all 1090 dimensions and with no knowledge of which the attacker moved; realization (b) then overlays the two purified quantities onto the original adversarial packets, preserving identity and payload content, and recomputes the dependent header fields. The packets leaving (b) are what the protected NIDS receives. Only (a) is attack-agnostic: the overlay in (b) is fixed to this threat model's two axes (Section 4.5.3).

4.5.1 Purification Model

The reconstructor shares the meta-learner's scaffold, the decoder-only block of Figure 2.3 in its pre-norm form, with a strict causal mask and sinusoidal positions, scaled up to three layers, four heads, and $d_{\text{model}} = 128$ (about 678K parameters), with the 1090-dimensional token as both input and output. Its head, however, does not predict the entire clean sequence directly: it predicts a per-dimension residual

Δ, and the output is the input plus Δ (Figure 4.6(a)). This residual formulation fits the problem. An adversarial example is a perturbation of a clean sample, and the manipulations of this threat model touch only two physical quantities, timing and size, so out of the 1090 dimensions of a token only a handful actually need correcting; with a residual head, the identity map is the easy default ($\Delta \approx 0$), the untouched ~ 1050 header bits are preserved, and the model's capacity is spent on the localized corrections that matter. The causal mask, in turn, keeps the purifier valid for streaming use and compatible with key–value caching (Section 2.1.3). The purifier is nonetheless not autoregressive, since it never feeds its own output back as input: it corrects every input position in a single pass, each from packets $\leq t$ alone, so the causal mask is there for streaming, not for generation.

4.5.2 Training on Matched Flow Pairs

Purification is learned from matched pairs. Because the attacker perturbs a flow without changing its five-tuple identity, an adversarial flow can be aligned with the clean flow it was derived from by matching canonical five-tuple keys. In practice, TANTGen also splits, merges, and drops flows, so only about a third of the adversarial flows find such a counterpart (Section 5.2); these form the (adversarial $\rightarrow$ clean) pairs on which reconstruction is supervised. Pairs shorter than two packets are dropped, since a single packet carries no inter-arrival interval and thus no timing signal to restore. Pairs follow the same seed-42 flow-level split as everything else, keeping the test pairs unseen by every base learner.

The loss is a smooth-L1 distance between the reconstructed and the clean sequence, masked over padding and averaged within each flow before averaging across flows, the same length-normalization argument as in Section 4.4.2; smooth-L1 is preferred over a plain squared error for its robustness to the occasional large deviation. One further choice is important: the loss weights all 1090 dimensions uniformly. This means that the training signal contains no hint of where TANTGen's perturbation lives, so the model must discover for itself which dimensions are trustworthy and which have been tampered with. This keeps the purification model attack-agnostic: nothing in its objective is specific to TANTGen's footprint, even though that footprint is known here. During development a variant weighted towards the known perturbed dimensions was also explored, and it was dropped from the final design because it ties the defense to one attacker's signature. This agnosticism is a property of this specific feature-space stage, not of the whole purification path, and Section 4.5.3 states where it stops.

4.5.3 Traffic-Space Realization and Validation

A purifier that edits traffic inherits the realizability obligation of Section 2.2.3: its output must be traffic that can exist on the wire. DEPT meets this obligation with an explicit realization step (Figure 4.6(b)) at the end of the inference path. Rebuilding packets from scratch out of the purified tensors would

be impossible, as nPrint encodes only headers and the identity fields are masked to zero. Realization therefore works by overlay: we keep each original packet, with its identity and payload content intact, and write back only the two quantities purification may have changed. The purified IPv4 total length is decoded from its sixteen header bits and applied by trimming the application payload, which is the inverse of the attacker's size perturbation; the purified inter-arrival time is de-normalized and applied to the packet's capture timestamp. The packets are then re-serialized with scapy, which recomputes the checksums and length fields that depend on the edit. Alignment between tensor rows and packets reuses the pipeline's canonical flow grouping, giving a one-to-one correspondence; any flow whose packet count fails to match its tensor length is skipped, counted, and reported at the end.

The overlay has a price: the deployed pipeline is not attack-agnostic end to end. The reconstructor of Section 4.5.2 corrects all 1090 dimensions, but realization writes back exactly two, the axes the threat model of Section 4.1 admits. An adversary that perturbed, for example, the Time-to-Live (TTL) or the TCP window would still be flagged and denoised in feature space, but no overlay rule exists for those fields, so the flow would reach the protected NIDS unrepaired and DEPT would degrade to detection alone. Deriving the fields to edit from the purifier's own residual instead of fixing them in advance is left as future work (Section 6.4).

Every realized packet is finally passed through the `tshark` dissector, and protocol compliance is scored as the fraction of packets parsed without a malformed-packet complaint. This check is the one part of the stage that stays off the inference path: it is a measurement, for the compliance figures of Section 5.6 and for forensics, not a runtime filter, so no packet waits on it. One caveat should be stated here: for application-layer text protocols, such as the SSDP `M-SEARCH` messages that dominate one source, a size edit necessarily corrupts the L7 grammar even when all IP and UDP fields remain consistent, so such packets cannot be faithfully realized by overlay at all. They are reported separately rather than scored as compliance failures, and Section 5.6 gives the full accounting.

Finally, since the realized packets are what the protected NIDS receives, purification is judged mainly through that NIDS rather than through DEPT's own detector: Kitsune [17] for the two Kitsune-native sources and TANTGen's own ML-NIDS flow classifiers for the eight CIC attacks, in every case the detector TANTGen calibrated that attack's evasion against (Section 5.7.1).

4.6 Implementation and Tools

DEPT is implemented in Python. The three sequence models (TCN, meta-learner, reconstructor) are built in PyTorch and the RF in scikit-learn. The nPrint encoding is reimplemented in Python on top of scapy, following the bit layout of the original tool [23]; writing the encoding into the pipeline avoids an external binary, lets identity masking happen inline, and keeps tensorization a single pass over each

capture. Scapy also drives packet parsing and the realization step, while `tshark`, `editcap`, and `tcpdump` handle dissection, time-window filtering of CIC-IDS2017, and the packet-capping of the large CIC-DDoS-2019 captures, respectively.

Table 4.3: Configuration summary of DEPT's four learned components. A step is a checkpoint for the Random Forest and a packet for the three sequence models; the forest additionally uses $\sqrt{d}$ feature sampling and `balanced_subsample` class weighting.

	RF	**TCN**	**Meta-learner**	**Reconstructor**
Input (per step)	24	1090	13	1090
Blocks / layers / trees	200 trees	3 temporal	2 causal	3 causal
Width / depth	max depth 20	128 channels	$d_{\text{model}} = 64$	$d_{\text{model}} = 128$
Attention heads	—	—	4	4
Feed-forward dim	—	—	128	256
Regularization	min leaf 5	dropout 0.15	dropout 0.1	dropout 0.1
Parameters	—	$805{,}762$	$67{,}971$	$677{,}698$
Loss / criterion	Gini impurity	balanced CE	balanced CE + aux	smooth-L1
Learning rate	—	10^{-3}	$3 \cdot 10^{-4}$	$3 \cdot 10^{-4}$
Batch size	—	32	128	32
Max epochs	—	20	20	20
Selection criterion	val macro-F_1	val macro-F_1	AUROC@10	val loss

The system is organized as four model stages behind five configuration files, one per stage plus one for the pipeline, with the three evaluation studies of Chapter 5 (the gradient-guided attack, the deployment baseline, and the comparison with a published purifier) adding one configuration file each in the same pattern; every file fully determines its stage's data scope, architecture, and training settings, and a small runner script per stage executes it end to end. All stages read the same tensor artifacts and the same stored split, which is how the shared-split principle of Section 4.3 is enforced in practice, and derived artifacts are cached. Table 4.3 summarizes all four learned components. The source layout mirrors the stages, with the feature pipeline under `src/feature_extraction`, the models under `src/models`, per-stage entry points under `scripts`, and configuration under `config`; further operational details live in the repository rather than in this document.

With the architecture in place, what remains is to show that it meets its goals: that the ensemble detects manipulation early, and that purification actually returns flagged flows to the benign manifold while emitting valid packets. These are the questions Chapter 5 takes up.

5

Evaluation

Contents

5.1 Research Questions 40

5.2 Experimental Setup 40

5.3 Metrics 41

5.4 Detection Results 41

5.5 Robustness under a Gradient-guided Attacker (RQ1) 45

5.6 Purification Results 47

5.7 End-to-End Evaluation 49

This chapter evaluates DEPT against the threat model of Section 4.1, on the corpus of Section 4.3. After the research questions, the setup and the metrics, it reports how well and how early DEPT detects manipulated flows, how much a gradient-guided attacker must spend to evade the ensemble, how much detection purification gives back to the protected NIDS, and what the full pipeline achieves end to end. The answers to the research questions and the limitations these results show are left to Chapter 6.

5.1 Research Questions

The evaluation is organized around two research questions, each tied to one of the work objectives of Section 1.3:

- **RQ1 (ensemble robustness).** Does heterogeneous stacking, a non-differentiable Random Forest fused with a differentiable Temporal Convolutional Network, measurably raise the cost of a transferable, gradient-guided evasion attack relative to a single-architecture baseline?
- **RQ2 (purification effectiveness).** Does the reconstructor move a flagged adversarial flow back onto the benign manifold, closing the detection gap seen by the protected NIDS, while emitting a realizable, protocol-compliant packet sequence?

Section 5.4 first reports the detection results, on which both questions build. TANTGen cannot answer RQ1, because it takes its gradients from its own surrogate and never from DEPT, so RQ1 uses the second adversary of Section 4.1, which differentiates the TCN directly (Section 5.5). RQ2 is answered on the reconstructor's test pairs (Section 5.6) and end to end, through the NIDS each attack was built to evade, where DEPT's purifier is also compared with a published one (Section 5.7). Whether the full pipeline meets a real-time latency budget is not evaluated in this work (Section 6.4).

5.2 Experimental Setup

The corpus has ten benign sources and thirty adversarial source–mode combinations, with $8{,}624{,}958$ flows and $61{,}516{,}406$ packets in total. The models are trained, validated and tested on the flow-level, source-stratified $70/15/15$ split of Section 4.3. The CIC-DDoS-2019 floods supply most of the adversarial flows, the SYN flood alone more than half, so the test set is mostly adversarial: of its $1{,}293{,}737$ flows, $1{,}082{,}586$ (83.7%) are adversarial and $211{,}151$ are benign. For this reason the models are compared with AUROC, which does not depend on the base rate. The Random Forest gives its first verdict at packet 2, so it does not score the $6{,}179$ single-packet test flows.

The reconstructor learns from pairs of an adversarial flow and its clean original. Of the $7{,}217{,}272$ adversarial flows, $2{,}566{,}489$ have such a counterpart; $1{,}796{,}542$ pairs are used for training and $384{,}963$ are held out for testing.

All experiments ran on one server with two Intel Xeon Gold 5320 CPUs (52 cores), 160 GB of RAM and two NVIDIA RTX 6000 Ada GPUs, using the software of Section 4.6. Generating the thirty adversarial captures with TANTGen took 26.7 h and tensorizing the corpus 11.2 h. Training took 2.7 h for the Random Forest, including its grid search, 24.7 h for the TCN, 3.4 h for the meta-learner and 10 h for the reconstructor.

5.3 Metrics

A – Detection. A defense that must act before a flow ends cannot be judged only at the end of the flow. The main detection metric is therefore AUROC@t, the ROC AUC computed with each flow's prediction after its first t packets, which shows how soon a model becomes useful; AUROC@MAX uses the prediction at the last packet. Two more metrics use the fixed threshold of 0.5: the Earliest Detection Packet (EDP), the median packet at which a flow's score first crosses the threshold, and the detection rate, the fraction of adversarial flows that cross it at some packet.

B – Purification. The reconstruction Mean Squared Error (MSE) compares a flow with its true clean counterpart, before and after purification, on the feature dimensions TANTGen perturbed. Since nPrint cells take the values -1, 0 and 1, a flipped bit costs 1 and a field that appears or disappears costs 4, so values above 1 are possible. The main purification metrics are measured through an independent NIDS, the detector each attack was built to evade (Section 5.7.1), so that no DEPT component grades itself. They are the Attack Success Rate (ASR), one minus the detection rate of that NIDS, and the restoration ratio, the fraction of the detection lost to TANTGen that purification recovers. The purification gain, the drop in the score of DEPT's own TCN from the adversarial to the purified flow, is only a sanity check, since that detector is part of DEPT. Protocol compliance is the fraction of realized packets that `tshark` parses without a malformed-packet error.

5.4 Detection Results

This section reports detection overall, per source and mode, and against per-attack models.

5.4.1 Streaming Detection

Table 5.1 and Figure 5.1 compare the three models on the streaming axis.

The Random Forest alone is the best model from packet 2 on. Its AUROC is 0.9805 at its first checkpoint and 0.9813 at the end of the flow, against 0.9099 to 0.9334 for the meta-learner and 0.8225 to 0.8386 for the TCN. The design expected the meta-learner to be at least as good as its best base learner, and it is not. It is the best model only at packet 1, where the forest has no verdict yet (0.8373 against 0.8191 for the TCN), and from packet 2 on it stays 8.7 to 9.5 points above the TCN. Its main contribution is robustness: under the gradient-guided attacker of Section 5.5, the fused stack is harder to evade than either base learner at low budgets.

At the 0.5 threshold the meta-learner is conservative. It flags only $1{,}115$ of the $211{,}151$ benign test flows, for a precision of 0.9985 on the adversarial class, and its EDP is the first packet.

Table 5.1: Streaming detection on the test flows. AUROC@t uses each flow's prediction after t packets; EDP is the median packet at which a flow is first flagged; the detection rate is the fraction of adversarial flows flagged at some packet. The Random Forest gives no verdict before packet 2.

Metric	RF	TCN	Meta-Learner
First verdict at	packet 2	**packet 1**	**packet 1**
AUROC@1	—	0.8191	**0.8373**
AUROC@2	**0.9805**	0.8225	0.9099
AUROC@5	**0.9805**	0.8375	0.9319
AUROC@10	**0.9813**	0.8371	0.9323
AUROC@MAX	**0.9813**	0.8386	0.9334
EDP (median)	2 pkts	**1 pkt**	**1 pkt**
Detection rate	**88.99%**	78.40%	77.69%

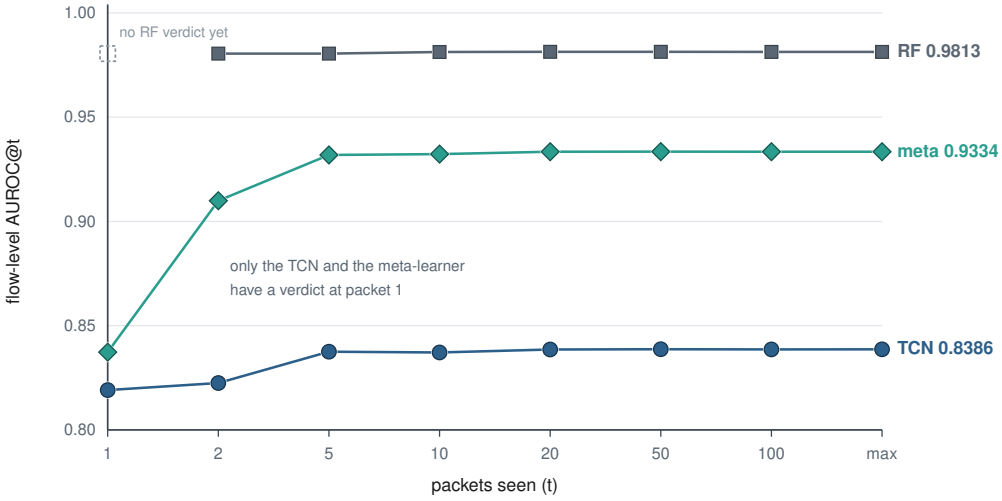


Figure 5.1: AUROC@t as a function of the packets seen. Only the TCN and the meta-learner have a verdict at packet 1; the Random Forest starts at packet 2, already at 0.98, and stays the best model after that.

On some sources the meta-learner falls far below the forest (Table 5.2). On `ntp_ddos both` and `iat` it reaches 0.653 to 0.654 where the forest reaches 0.999, and on `ssdp_ddos` it reaches 0.927 against 1.000. A likely cause is the one anticipated in Section 4.4.3. The forest's logit enters the fusion token as a single feature, updated only at checkpoints and missing before packet 2, while the TCN's logit changes at every packet, so attention gives the forest too little weight. The auxiliary "beat the best base" loss helps only in part. It falls from 0.0267 to 0.0177 during training, so the meta-learner does move towards its better base learner, and the validation AUROC@10 rises from 0.9197 to 0.9333, mostly in the first epochs, but the forest stays ahead.

5.4.2 Detection per Source and Mode

The aggregate AUROC hides large differences between sources. Table 5.2 gives the end-of-flow AUROC of the 30 source–mode combinations, and Figure 5.2 plots the same values. Each combination is

Table 5.2: End-of-flow AUROC per source and perturbation mode (`b`, `i`, `s`: `both`, `iat`, `size`), against benign flows of the same source. n_{adv} is the number of adversarial test flows per mode ($4{,}842$ for `ddos size`). The Random Forest scores the same flows, except on `mirai`, where many flows end before packet 2 and it scores only 867 of the $1{,}957$, and on `ssdp_flood` ($9{,}830$). Values below 0.55 are marked with †.

Source	n_{adv}	M.	RF	TCN	Meta
		b	0.974	0.978	0.981
`brute_force`	$1{,}045$	i	0.974	0.978	0.979
		s	0.990	0.978	0.989
		b	1.000	1.000	1.000
`ddos`	$5{,}781$	i	0.999	0.924	0.967
		s	1.000	1.000	1.000
		b	0.999	0.552	1.000
`dns_ddos`	$1{,}019$	i	0.999	0.547†	1.000
		s	0.258†	0.531†	0.443†
		b	1.000	0.500†	1.000
`ldap_ddos`	$74{,}710$	i	1.000	0.500†	1.000
		s	0.476†	0.501†	0.476†
		b	0.926	0.680	0.724
`mirai`	$1{,}957$	i	0.919	0.680	0.719
		s	0.829	0.723	0.734

Source	n_{adv}	M.	RF	TCN	Meta
		b	0.999	0.514†	0.654
`ntp_ddos`	$2{,}103$	i	0.999	0.497†	0.653
		s	0.213†	0.494†	0.493†
		b	1.000	0.997	0.998
`portscan`	$23{,}800$	i	1.000	0.652	0.757
		s	0.998	0.997	0.997
		b	1.000	0.499†	0.927
`ssdp_ddos`	$40{,}446$	i	1.000	0.502†	0.927
		s	0.409†	0.497†	0.497†
		b	1.000	0.999	1.000
`ssdp_flood`	$9{,}831$	i	1.000	1.000	1.000
		s	1.000	0.999	1.000
		b	1.000	1.000	1.000
`syn_ddos`	$200{,}483$	i	1.000	0.647	0.806
		s	1.000	1.000	1.000

measured against benign flows of the same source, since DEPT must tell a manipulated flow from an unmanipulated flow of the same kind.

Most combinations are solved: 21 of the 30 reach at least 0.99 with the Random Forest, and 13 with the meta-learner. The weak spots are of four kinds.

Timing-only edits are hard for the TCN and easy for the forest. On `portscan_iat` the TCN reaches 0.652 and the forest 1.000, on `syn_ddos_iat` 0.647 and 1.000, and on `ddos_iat` 0.924 and 0.999. The forest's timing statistics capture a change in timing that the TCN's per-packet view misses, which is the kind of difference between learners that the ensemble was built on.

The TCN fails on the four reflection floods. It stays between 0.494 and 0.552 in every mode of `dns_ddos`, `ldap_ddos`, `ntp_ddos` and `ssdp_ddos`, and on `ldap_ddos`, the largest of them, it gives almost exactly 0.500 in all three modes. These floods are one- and two-packet UDP exchanges, and a receptive field of 29 packets has nothing to convolve over.

The `size` variants of the same four floods are at or below chance for every model: 0.213 to 0.476 for the forest, 0.443 to 0.497 for the meta-learner and 0.494 to 0.531 for the TCN. Below chance means that, on these variants, TANTGen's flows look more like generic benign traffic than the flood's own unedited flows do. Section 5.6 shows that on three of them the edit did not change the flows at all.

Finally, `mirai` is the only source that no model solves in any mode, with a best value of 0.926, but no model falls below chance on it either.

5.4.3 One Global Model or Ten Specialists?

All results so far come from one set of models trained on the ten sources together. Detectors in the literature are often trained per attack instead, so the four model stages were also run once per attack,

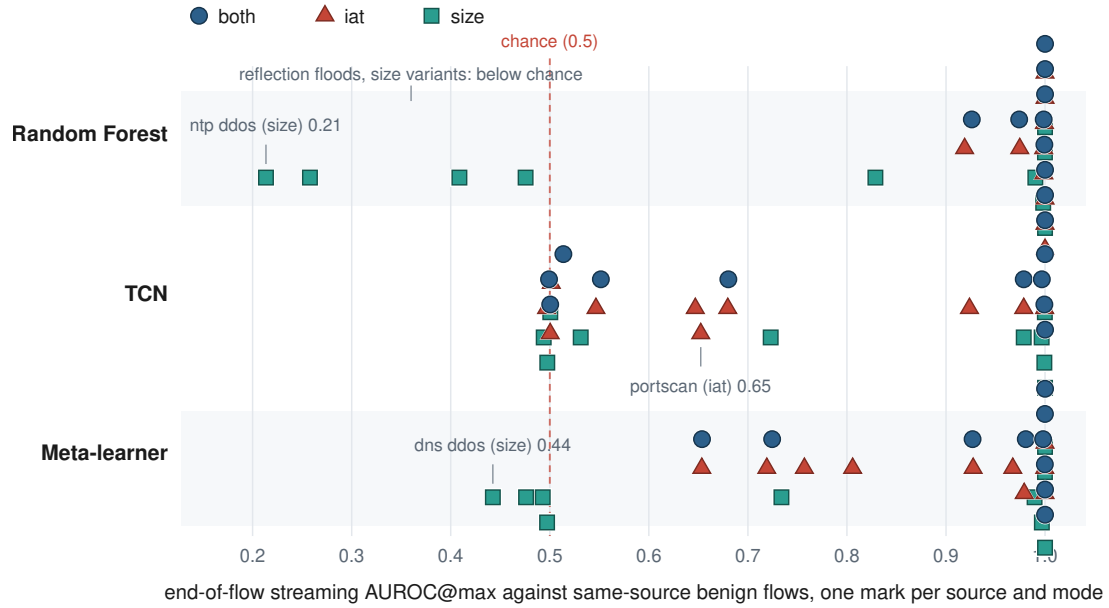


Figure 5.2: The values of Table 5.2, one mark per source–mode combination and one row per model. Most combinations are solved; the `size` variants of the reflection floods sit below chance.

Table 5.3: Meta-learner AUROC against same-source benign flows, trained on all ten sources (global) or on that attack alone (spec.). Δ is specialist minus global; below 0.005 it is shown as ≈ 0.

	both			iat			size		
Source	spec.	global	Δ	spec.	global	Δ	spec.	global	Δ
`brute_force`	0.995	0.981	$+0.014$	0.995	0.979	$+0.016$	0.997	0.989	$+0.007$
`ddos`	1.000	1.000	≈ 0	1.000	0.967	$+0.033$	1.000	1.000	≈ 0
`dns_ddos`	1.000	1.000	≈ 0	1.000	1.000	≈ 0	0.192	0.443	-0.251
`ldap_ddos`	1.000	1.000	≈ 0	1.000	1.000	≈ 0	0.435	0.476	-0.041
`mirai`	0.689	0.724	-0.035	0.687	0.719	-0.033	0.740	0.734	$+0.006$
`ntp_ddos`	1.000	0.654	$+0.346$	1.000	0.653	$+0.347$	0.313	0.493	-0.180
`portscan`	0.998	0.998	≈ 0	0.824	0.757	$+0.066$	0.996	0.997	≈ 0
`ssdp_ddos`	1.000	0.927	$+0.073$	1.000	0.927	$+0.073$	0.355	0.497	-0.142
`ssdp_flood`	1.000	1.000	≈ 0	1.000	1.000	≈ 0	1.000	1.000	≈ 0
`syn_ddos`	1.000	1.000	≈ 0	1.000	0.806	$+0.194$	1.000	1.000	≈ 0

with the same architectures, grid searches and split protocol. Table 5.3 compares each specialist meta-learner with the global one.

The two are indistinguishable on 13 of the 30 combinations, all of them already at or near 1.000. The specialists are better on timing edits: `ntp_ddos` gains 0.346 and 0.347 on `both` and `iat`, `syn_ddos_iat` 0.194, `ssdp_ddos` 0.073 on both modes and `portscan_iat` 0.066. A model trained on one attack can learn that source's own timing, instead of mixing it with nine other benign sources. The global model is better on `mirai` `both` and `iat` (by 0.035 and 0.033) and on the `size` variants of the reflection floods. There, however, both are below chance, and the global model is only less wrong (0.443 against 0.192 on `dns_ddos`).

The specialists also show that two problems of the global model do not come from training on all

sources at once. The meta-learner again falls below its own forest, most clearly on `mirai` (0.705 against 0.889) and on `portscan` (0.939 against 0.999). The per-attack TCNs also fail on the reflection floods, at 0.500 to 0.531, while doing well on sources with longer flows (0.990 on `ddos` and 1.000 on `ssdp_flood`).

A global model is still the better default, because a specialist can only be chosen by a router that first names the attack, which is the classification problem the detector is there to solve. Specialists are worth considering only for a known flood whose evasion is in the timing.

5.5 Robustness under a Gradient-guided Attacker (RQ1)

The detection results above show that the ensemble detects TANTGen's manipulations, but not what it is worth against an attacker who can differentiate one of its members. This section answers RQ1 with the second adversary of Section 4.1.

5.5.1 Attack Setup

The attacker has white-box access to the TCN and none to the Random Forest. It crafts one perturbation against the TCN with PGD [37], and the same perturbation is then given, unchanged, to the TCN, the Random Forest and the fused stack. Optimizing it again for each target would no longer measure transfer. The perturbation adds padding and delay to each packet, on top of TANTGen's, and never shortens a packet or makes it longer than the Maximum Transmission Unit (MTU). The padding bytes are random. The TCN does not see payload bytes, so its gradient cannot choose them, but their content changes the payload entropy that the forest reads. The budget is set per packet, from 8 bytes and $10\,\mu$s to $1{,}460$ bytes and 100 ms.

A defense tested only with a weak attack can look more robust than it is [67, 68], so two other searches check the PGD attacker. A random search uses the same constraints, budget and number of queries, but no gradient. AutoPGD [69], from the Adversarial Robustness Toolbox (ART) library [70], checks that PGD is not badly tuned: on a relaxed version of the problem (sixteen `brute_force` flows, without integer or budget limits), it and both of our searches evade the TCN on every flow. The reported attacker takes, for each flow, the better of PGD and random search.

The attack runs on $9{,}000$ test flows, 300 per source–mode combination. Each target's threshold is set at a 5% false-positive rate on benign traffic, because the three models are calibrated very differently: their median verdict on benign traffic is 0.043 for the forest, 0.379 for the TCN and 0.002 for the stack, and the matched thresholds are 0.4535, 0.9820 and 0.3658. The ASR is counted only over the flows a target detects without perturbation, since a flow that is not detected cannot be evaded. An attacker with a larger budget may also spend less, so the success rate at a budget includes the flows evaded at every smaller budget.

5.5.2 Results

Table 5.4: Attack success rate at a matched 5% false-positive rate, for one perturbation crafted on the TCN and replayed on all three targets. The budget is a cap per packet; the spend and delay columns are medians of what the attacker actually used. Success is counted over the flows each target detects without perturbation: $7{,}510$ for the RF, $3{,}374$ for the TCN and $7{,}597$ for the stack, of $9{,}000$.

Budget (B/pkt)	**Spend** (B/pkt)	**Delay** (ms/pkt)	**Spend** (B/flow)	**RF**	**TCN**	**Stack**
$\delta = 0$	0	0	0	0.0%	0.0%	0.0%
8	3.0	0.004	13	8.5%	40.5%	**1.7%**
32	14.0	0.041	47	10.7%	50.1%	**4.7%**
128	49.5	0.419	167	13.1%	52.2%	**8.5%**
512	159.8	3.677	537	**18.4%**	53.3%	18.9%
1460	402.0	28.513	1,574	36.0%	54.5%	**25.5%**
median budget to evade (B/pkt)				512	8	512
never evaded, any budget				64.0%	45.5%	**74.5%**

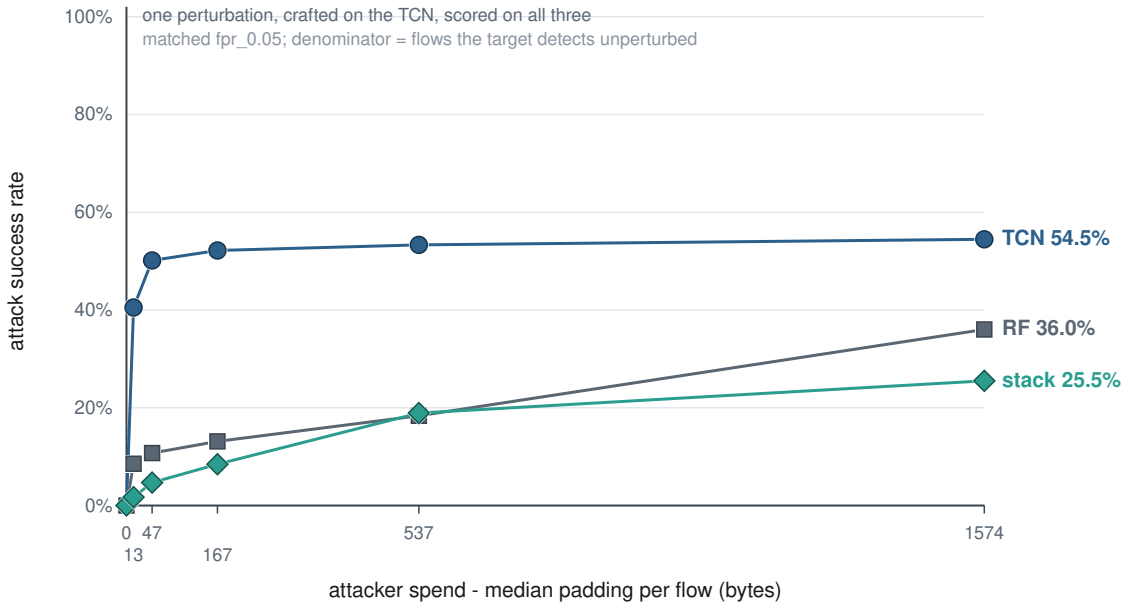


Figure 5.3: Attack success on each target as the attacker's budget grows, for one perturbation crafted with the TCN's gradients and replayed unchanged on the other two targets, at a matched 5% false-positive rate.

A – The stack is the hardest target at low budgets. Table 5.4 and Figure 5.3 answer RQ1. At 8, 32 and 128 bytes per packet, the stack concedes 1.7%, 4.7% and 8.5% attack success, against 8.5%, 10.7% and 13.1% for the forest and 40.5%, 50.1% and 52.2% for the TCN. Over the whole grid, 74.5% of the flows never evade the stack, against 64.0% for the forest and 45.5% for the TCN. The perturbation that breaks the TCN transfers poorly to the forest, and the stack is harder still, although not at every budget: at 512 bytes the stack (18.9%) and the forest (18.4%) are level, and at $1{,}460$ bytes the stack is ahead again

Table 5.5: Purification results on the $384{,}963$ test pairs. The reconstruction MSE is computed on the perturbed dimensions; the TCN row is only a sanity check; compliance counts realizable packets only.

Quantity	Value
Reconstruction MSE (adversarial $\rightarrow$ purified)	$0.918 \rightarrow \mathbf{0.054}$ $(17.0\times)$
Independent NIDS detection (adversarial $\rightarrow$ purified)	$0.781 \rightarrow \mathbf{0.906}$
Detection of the same NIDS on the clean flows	0.980
Restoration ratio	$\mathbf{63.0}\%$
TCN $P(\text{adv})$ (adversarial $\rightarrow$ purified; clean)	$0.675 \rightarrow 0.428$; 0.421
`tshark` pass rate ($2{,}412{,}357$ realizable packets)	$\mathbf{97.22}\%$
Packets not realizable at L7 (reported apart)	$450{,}758$

(25.5% against 36.0%). For comparison, TANTGen's own padding averages 29 to 32 bytes per packet on `portscan_both` and `mirai_size`, but 246 to 289 bytes on the `size` variants of `ddos`, `ssdp_flood` and `portscan`, closer to the 512-byte point.

B – The meta-learner leans on the forest under attack. If the ensemble works as intended, the meta-learner should rely more on the forest when the TCN is attacked. To check this, each base-learner slot of the fusion token is given back its clean value, and the change in the stack's verdict is measured. The share of that change due to the forest is 0 without attack, since there is nothing to compensate, 0.529 as soon as the attacker spends anything, and between 0.54 and 0.56 at the larger budgets.

Against the TCN itself, random search does better than PGD at every budget (40.1% against 37.2% at 8 bytes, 52.8% against 50.7% at $1{,}460$). Neither wins on every flow, so the best-of attacker is stronger than both (40.5% and 54.5% at the same budgets). In practice, the attacker measured here is a search attack, and the gradient adds little to it. The transfer result still holds, since both searches run on the TCN.

Two more limits apply to this result. The TCN's numbers rest on $3{,}374$ of the $9{,}000$ flows, because at its threshold it detects at most 2% of the unperturbed flows on four sources (`brute_force`, `ldap_ddos`, `ntp_ddos` and `ssdp_ddos`) and 9% on `mirai`. And on the `both` and `size` modes of `ddos` and `portscan`, and on `ssdp_flood_size`, neither the forest nor the stack is evaded at any budget, which only shows that padding and delay cannot overcome that much model confidence.

5.6 Purification Results

The reconstructor is evaluated on the $384{,}963$ held-out test pairs. Every purified flow is realized into packets (Section 4.5.3), and the compliance and NIDS figures below are measured on those packets. Table 5.5 gives the overall results.

On the dimensions TANTGen perturbed, the reconstruction error falls from 0.918 to 0.054, a $17.0\times$

Table 5.6: Purification by source (representative rows): reconstruction MSE on the perturbed dimensions, before and after purification, and detection by the independent NIDS on the adversarial and purified flows, with the clean flows in parentheses. A dash marks a value the run does not record: `ldap_ddos size` has no perturbed cells, and no packet of `ssdp_flood size` is realizable. Only 24 packets of `ssdp_flood both` are realizable, and the Kitsune rows (`mirai`, `ssdp_flood`) are scored on test flows only.

Source	Mode	MSE (before → after)	NIDS det. adv → pur. (clean)
Footprint reduced, detection restored			
`ddos`	size	$1.000 \to 0.117$	$0.005 \to 0.817$ (0.828)
`ddos`	both	$0.928 \to 0.124$	$0.080 \to 0.865$ (0.821)
`syn_ddos`	size	$1.000 \to 0.144$	$0.879 \to 1.000$ (1.000)
`syn_ddos`	both	$0.973 \to 0.129$	$0.973 \to 0.999$ (0.998)
`portscan`	both	$0.972 \to 0.105$	$0.011 \to 0.679$ (0.999)
`portscan`	size	$1.000 \to 0.104$	$0.013 \to 0.479$ (0.999)
`ssdp_flood`	iat	$0.110 \to 0.081$	$0.001 \to 1.000$ (1.000)
`ssdp_flood`	both	$0.965 \to 0.003$	$1.000 \to 1.000$ (1.000)
`ssdp_flood`	size	$1.000 \to 0.003$	—
Nothing to undo			
`ldap_ddos`	both, iat	$\leq 0.18 \to \approx 0$	$1.000 \to 1.000$ (1.000)
`ldap_ddos`	size	—	$1.000 \to 1.000$ (1.000)
`ssdp_ddos`	both, iat	$\approx 0 \to \approx 0$	$0.998 \to 0.998$ (0.998)
`ntp_ddos`	both, iat	$\approx 0 \to \approx 0$	$0.983 \to 0.986$ (0.998)
Detection lowered where TANTGen did not evade			
`dns_ddos`	iat	$\approx 0 \to \approx 0$	$0.993 \to 0.756$ (0.998)
`dns_ddos`	both	$\approx 0 \to \approx 0$	$0.986 \to 0.762$ (0.996)
`dns_ddos`	size	$1.000 \to 0.302$	$0.986 \to 0.765$ (0.986)
`portscan`	iat	$\approx 0 \to 0.007$	$0.994 \to 0.811$ (0.999)
`brute_force`	size	$1.000 \to 0.188$	$0.993 \to 0.963$ (0.994)
`mirai`	size	$1.569 \to 0.973$	$0.257 \to 0.005$ (0.008)

reduction. Through the independent NIDS, over $370{,}700$ realized flows, purification raises detection from 0.781 to 0.906, against 0.980 on the clean flows, and so recovers 63.0% of the detection TANTGen had removed. DEPT's own TCN agrees: it scores the purified flows at 0.428, close to the 0.421 of the true clean flows.

The overall figure mixes two detectors. On the eight ML-NIDS sources, detection rises from 0.808 to 0.910, against 0.987 on the clean flows, an average diluted by the many sources where detection was never lost. On the two Kitsune sources it rises from 0.019 to 0.792, against 0.793, but this figure means little. Most of these flows ($9{,}830$ of $12{,}417$) belong to `ssdp_flood_iat`, and Kitsune is partly volumetric, so its detection rate is only meaningful on full streams, such as those of Section 5.7.1. There, this variant shows no evasion at all.

The effect depends strongly on the source (Table 5.6).

Where TANTGen's edits were structural, purification removes most of the footprint, and where they had evaded, detection comes back: on `ddos_size` from 0.005 to 0.817 (clean: 0.828), on `ddos_both` from 0.080 to 0.865, on `portscan_both` from 0.011 to 0.679 and on `portscan_size` from 0.013 to 0.479. On `ssdp_flood_size` the footprint is also removed, but none of its packets can be realized, so the NIDS cannot score it.

On the `both` and `iat` variants of the four reflection floods there is nothing to undo: TANTGen barely

moved the features, and the footprint is close to zero before purification. On the `size` variants of `ldap_ddos`, `ntp_ddos` and `ssdp_ddos` the edit did not happen at all. Every adversarial tensor is identical to its clean counterpart, so there is no footprint to measure, and detection does not change.

Where TANTGen did not evade, purification can lower detection. On `dns_ddos` it falls from 0.986 to 0.762 on `both`, from 0.993 to 0.756 on `iat` and from 0.986 to 0.765 on `size`. It also falls on `portscan_iat`, from 0.994 to 0.811, and on `brute_force_size`, from 0.993 to 0.963. The purifier moves these flows towards their clean form, as it was trained to do, but here the manipulated and the clean flows were already close, so it removes information the NIDS was still using. On `dns_ddos_size` the footprint falls from 1.000 to 0.302, yet DEPT's own TCN scores the purified flows like the adversarial ones, because it cannot tell this source's adversarial and clean flows apart. `mirai` is where purification works least well: on its `size` variant the footprint only falls from 1.569 to 0.973, and detection drops to about its level on the clean flows, because TANTGen's padding had made this traffic easier for Kitsune to detect.

Of the realizable packets, 97.22% parse in `tshark` without errors. This rate leaves out $450{,}758$ packets of `ssdp_flood` `both` and `size`. A size edit on these SSDP/HTTP `M-SEARCH` text messages breaks their application-layer grammar even when the IP and UDP fields stay valid, so they cannot be realized faithfully and are reported apart. Among the realizable packets, compliance is below 0.99 only on `ddos` (0.84) and on `syn_ddos_iat` (0.178).

Figure 5.4 follows one test flow through the pipeline, a four-packet Mirai SYN-flood burst under the `size` variant. Each original packet has 44 bytes, a 20-byte IPv4 header and a 24-byte TCP header with no payload. TANTGen appends 29 bytes of payload to every packet, which raises the IPv4 total length to 73 bytes, and this is the whole perturbation, since the `size` variant does not change timing. The reconstructor predicts the original length, realization trims the added bytes, and every packet leaves with 44 bytes again. All twelve packets of the three captures pass `tshark` without errors.

5.7 End-to-End Evaluation

Section 5.6 purified every test pair. This section first measures DEPT as it would be deployed, with only the flows flagged by the meta-learner purified and complete streams given to the protected NIDS. It then compares DEPT's purifier with a published one, on the test pairs of Section 5.6. Since the two parts use different sets of flows, their numbers should only be compared within each part.

5.7.1 Detect-then-Purify Pipeline

Here the full gated pipeline of Section 4.2 is placed in front of the protected NIDS: the meta-learner flags flows, only the flagged flows are purified, and the rest pass unchanged. Each attack is scored through the NIDS it was built to evade, the same model TANTGen calibrated its evasion against. This

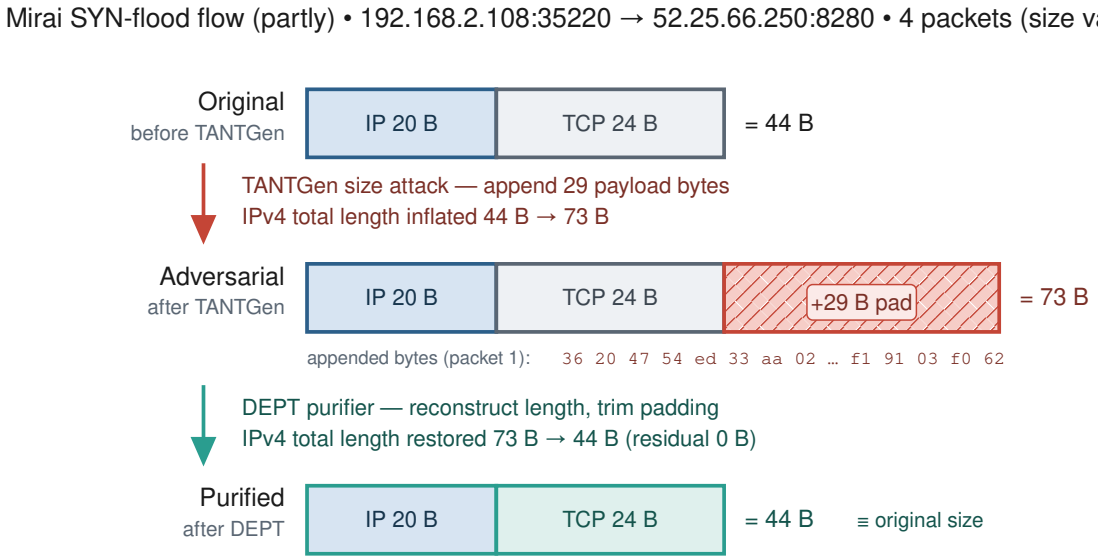


Figure 5.4: A real packet from the held-out test split through the purification pipeline: the $44 \rightarrow 73 \rightarrow 44$ byte restoration described in the text.

is Kitsune (KitNET + AfterImage) for the two Kitsune sources, and TANTGen's ML-NIDS, its per-attack CICFlowMeter flow classifiers, for the eight CIC-IDS2017 and CIC-DDoS-2019 attacks. For every attack, three full streams are realized and scored: the clean attack, which shows the NIDS working as intended, the adversarial traffic before DEPT, and the defended traffic after DEPT. The three streams share the same packets and differ only in sizes and timings (Section 4.5.3). Full streams are used because Kitsune, being partly volumetric, needs complete traffic. Table 5.7 and Figure 5.5 give the results.

Over $2{,}704{,}816$ scored flows, DEPT lowers the attack success from 21.1% to 11.6%, a reduction of 9.5 points, and recovers 49.3% of the detection TANTGen had removed. This average mixes very different outcomes. The gain is large where TANTGen evaded: 38 points on `ddos` and 32 on `portscan`, and on single modes 67.4 points on `ddos_size`, 67.0 on `portscan_both`, 59.7 on `ddos_both` and 46.8 on `portscan_size`. These are the structural edits that purification undid in Section 5.6. On `ldap_ddos`, `ntp_ddos`, `ssdp_ddos` and `brute_force`, TANTGen never really evaded, since the adversarial traffic was already detected at 0.98 to 1.00, and the pipeline changes the result by less than a point on the three floods and by 1.2 points on `brute_force`. On other combinations the defense makes things worse: `dns_ddos` loses 22.5 points on `both` and 22.7 on `iat`, `portscan_iat` 18.2, `ddos_iat` 12.2 and `mirai_size` 23.9. On `mirai_size`, TANTGen's padding had raised Kitsune's detection from 0.003 to 0.242, and by removing the padding DEPT also removed that gain.

The gate does not prevent these losses. It flags almost all flood traffic, except on the `size` variants of the four reflection floods, where it flags almost nothing (0.0% on `ssdp_ddos`, 0.2% on `ntp_ddos`, 0.8% on `ldap_ddos` and 0.9% on `dns_ddos`), the same blind spot as in Table 5.2. Flows that the NIDS was still

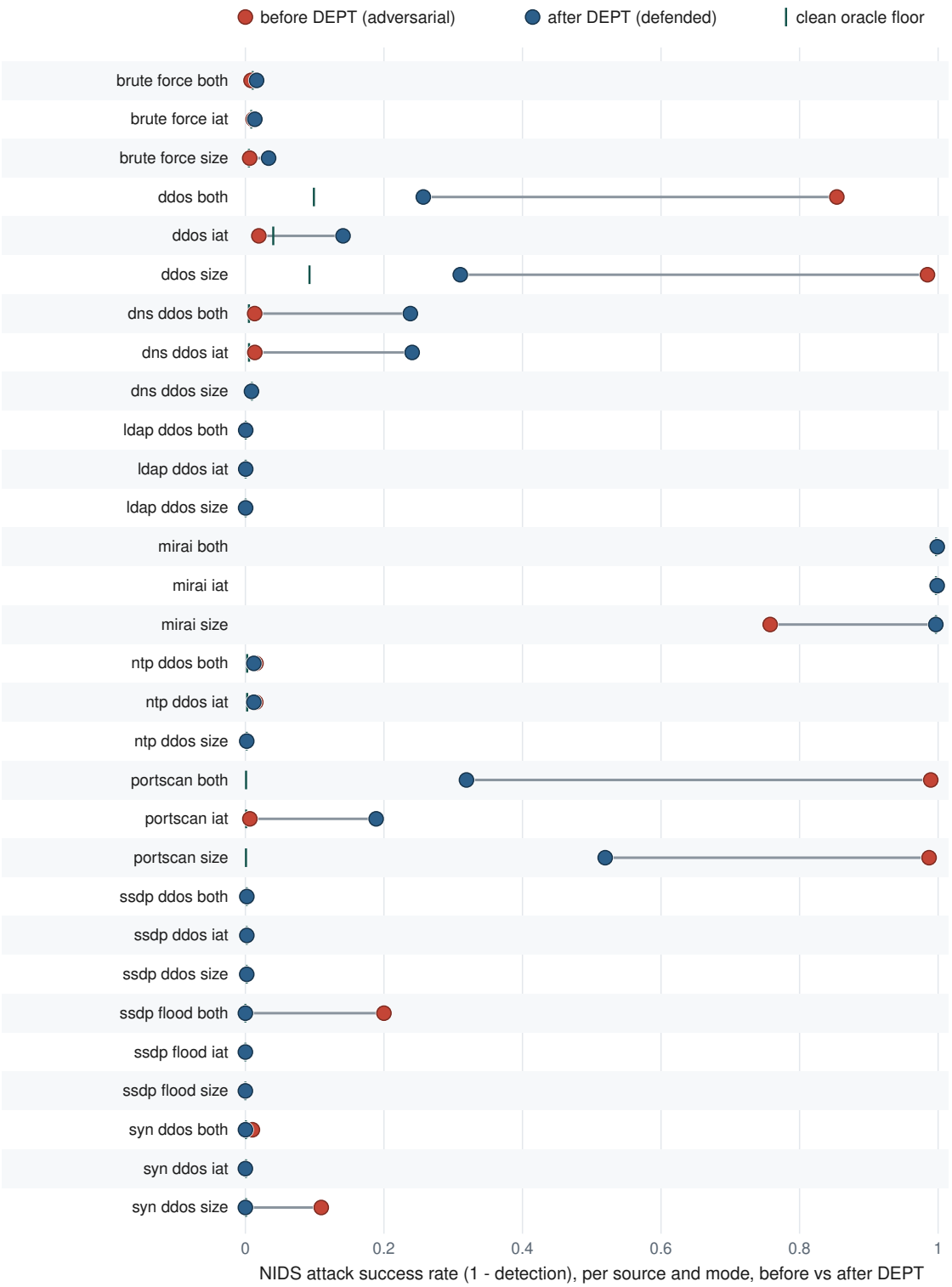


Figure 5.5: Attack success rate of the protected NIDS per source and mode, before and after DEPT, with the clean attack marked. On `ssdp_flood` `both` and `size` only five flows keep realizable packets, so the change on `both` is a single flow.

Table 5.7: Deployment baseline: attack success rate (ASR, one minus the detection rate) of the protected NIDS on the clean attack, on TANTGen's traffic (before DEPT) and on the output of the gated pipeline (after DEPT), per attack over its three perturbation modes, weighted by flow count. Δ is the reduction in ASR; positive is better.

Attack	NIDS	ASR clean	ASR before	ASR after	Δ
ddos	ML-NIDS	0.078	0.616	0.234	$+0.382$
portscan	ML-NIDS	0.001	0.661	0.342	$+0.319$
syn_ddos	ML-NIDS	0.001	0.047	0.000	$+0.047$
ntp_ddos	ML-NIDS	0.002	0.011	0.009	$+0.002$
ldap_ddos	ML-NIDS	0.000	0.000	0.000	$+0.000$
ssdp_ddos	ML-NIDS	0.002	0.002	0.002	$+0.000$
brute_force	ML-NIDS	0.008	0.009	0.021	-0.012
dns_ddos	ML-NIDS	0.006	0.013	0.192	-0.179
mirai	Kitsune	0.997	0.918	0.998	-0.080
ssdp_flood	Kitsune	0.000	0.000	0.000	$+0.000$
Overall		$\mathbf{0.019}$	$\mathbf{0.211}$	$\mathbf{0.116}$	$\mathbf{+0.095}$

catching are therefore purified too, and purifying every flow gives almost the same detection as the gated pipeline (0.8831 against 0.8836). The gate predicts whether a flow is adversarial, while the pipeline needs to know whether purifying it will help, and the results above show that these are different questions.

Because full streams are used, the defended streams also contain flows that the gate and the purifier saw in training. On the flows that neither saw in training, selected as in Section 5.7.2, the gated pipeline lowers the attack success from 31.4% to 17.7%, so the training overlap does not inflate the result.

5.7.2 Comparison with a Published Purifier

To compare DEPT with the state of the art, its purifier is replaced by the diffusion-based purifier of Merzouk et al. [2], the only network purifier of Section 3.3 with public code, while the flows, the realization step and the protected NIDS stay the same. The baseline keeps the authors' model and settings, learns only from clean traffic, and its amount of noise is chosen as in the original, on validation flows. Both purifiers are scored on the reconstructor's held-out test pairs (Table 5.8 and Figure 5.6). Overall, the attack success is 31.5% without defense, 18.9% with DEPT and 40.3% with the diffusion purifier, and the order is the same on test flows whose original flow appears in no training pair.

The comparison favours DEPT in two ways. First, DEPT learns from matched TANTGen and clean flows, while the baseline only sees clean traffic, and both are tested against TANTGen, the attacker DEPT was trained on. Second, the baseline was designed for flow-level features and had to be adapted to DEPT's per-packet sizes and timings; the timing encoding chosen for it explains part of its losses on the floods whose detection depends on timing.

Table 5.8: Comparison with the diffusion purifier of Merzouk et al. [2]: attack success rate (ASR) of the protected NIDS on the held-out test pairs, for the clean attack, for TANTGen's traffic without defense, and after each purifier, aggregated over the three perturbation modes. On `ssdp_flood`, only the `iat` mode keeps realizable packets, 2% of them.

Attack	NIDS	ASR clean	No defense	Diffusion	DEPT
`ddos`	ML-NIDS	0.210	0.684	0.893	0.357
`portscan`	ML-NIDS	0.002	0.661	0.836	0.343
`syn_ddos`	ML-NIDS	0.002	0.047	0.391	0.000
`ntp_ddos`	ML-NIDS	0.001	0.011	0.044	0.013
`ldap_ddos`	ML-NIDS	0.000	0.000	0.000	0.000
`ssdp_ddos`	ML-NIDS	0.014	0.014	0.015	0.015
`brute_force`	ML-NIDS	0.006	0.004	0.008	0.016
`dns_ddos`	ML-NIDS	0.010	0.013	0.017	0.240
`mirai`	Kitsune	0.996	0.913	0.984	0.998
`ssdp_flood`	Kitsune	0.998	0.999	1.000	0.999
Overall		$\mathbf{0.057}$	$\mathbf{0.315}$	$\mathbf{0.403}$	$\mathbf{0.189}$

A – Where TANTGen evaded, DEPT recovers detection and the diffusion purifier does not. On the five source–mode combinations where TANTGen lowered detection by more than five points (`ddos` and `portscan` in the `both` and `size` modes, and `syn_ddos_size`), DEPT recovers 63.5% of the lost detection and the diffusion purifier 0.5%, and no amount of noise did better on the validation flows. A padded or delayed TANTGen flow still looks like realistic traffic, so a purifier that only knows clean traffic has little reason to change it.

B – Elsewhere, both purifiers can lower detection. The diffusion purifier slows down the floods whose detection depends on timing, raising the attack success on `syn_ddos` from 4.7% to 39.1%. DEPT is worse than the diffusion purifier on the three modes of `dns_ddos`, on `brute_force_both` and on `mirai_size`, where the diffusion purifier barely changes the traffic, and fewer of its packets are protocol-compliant on `ddos_both` (89.2% against 98.6%). The Kitsune rows say little: Kitsune detects almost none of the `mirai` traffic even when it is clean.

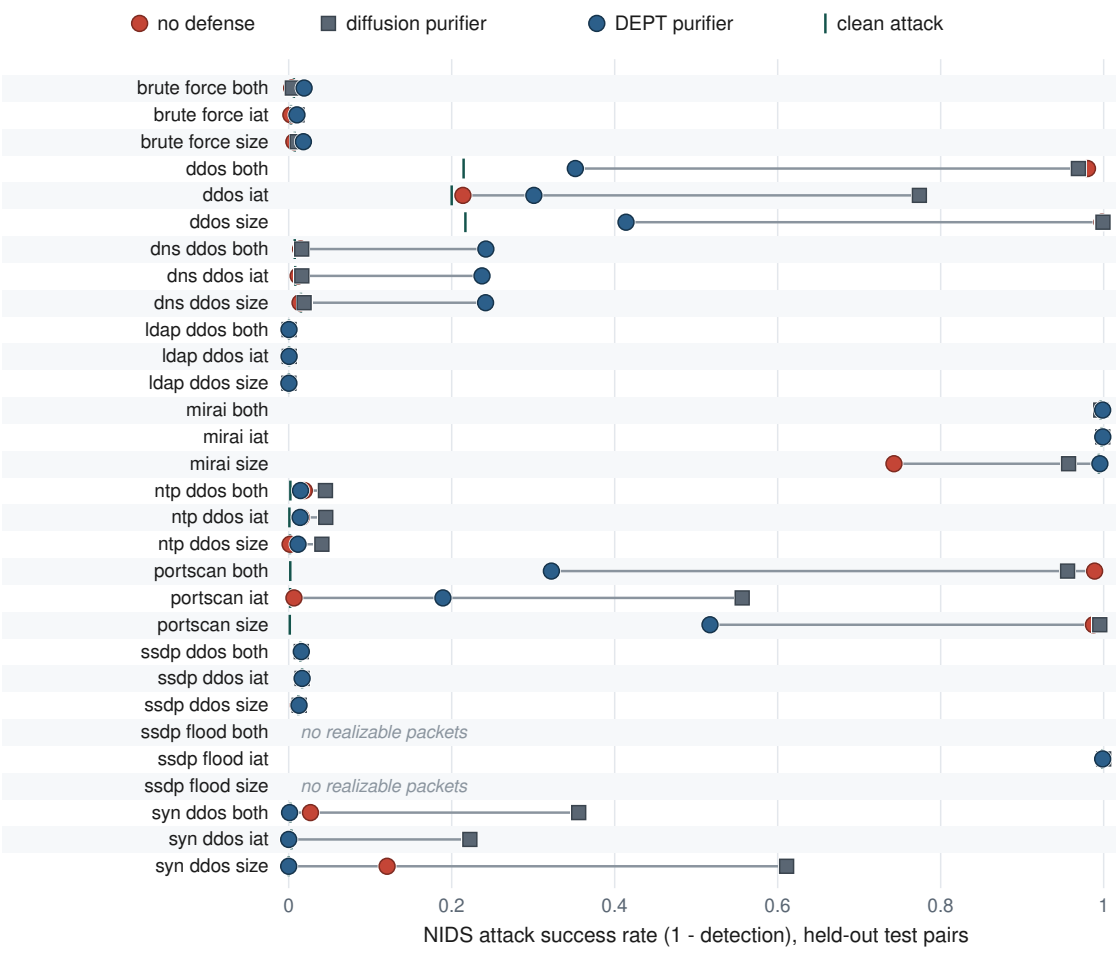


Figure 5.6: Attack success rate of the protected NIDS on each source and perturbation mode of the held-out test pairs, without defense and after each purifier, with the clean attack marked.

6

Conclusion

Contents

6.1 Summary of Contributions 55
6.2 Answers to Research Questions 56
6.3 System Limitations 57
6.4 Future Work 58

This thesis set out to defend a learning-based NIDS against a black-box attacker that reshapes live attack traffic into protocol-compliant flows the detector labels benign. Its answer is DEPT, a streaming defense that detects the manipulation mid-flow and undoes it before the protected system consumes it.

6.1 Summary of Contributions

The five contributions of Section 1.4 stand as follows. The packet-to-tensor pipeline (Section 4.3) turns raw captures into a masked, bit-level view of each flow with identity fields removed, dense per packet for the TCN and sparse statistics for the RF, over one flow-level split that makes the comparisons of Chapter 5 fair. The heterogeneous ensemble (Section 4.4) fuses the two at every packet through an attention-based meta-learner, scored on how early, not only how well, it separates manipulated from

clean flows; the purification stage (Section 4.5) pairs a causal Transformer predicting a residual correction with a realization step that writes it back into dissector-validated packets, so the protected NIDS receives traffic rather than tensors. The last two are the evaluation itself: the deployment baseline of Section 5.7.1, which judges DEPT through the real NIDS each attack was built to evade and compares its purifier with a published one (Section 5.7.2), and a reusable ten-source corpus in three perturbation modes, reported per source and per mode.

6.2 Answers to Research Questions

RQ1 (ensemble robustness), from Section 5.1, asked whether heterogeneous stacking measurably raises the cost of a transferable, gradient-guided evasion attack. It does. Since TANTGen follows only its own surrogate's gradients, the evaluation used a stronger adversary (Section 5.5): white-box access to the TCN, nothing for the RF and meta-learner, one perturbation against all three at once. It transfers poorly to the forest, and the stack is costlier still: at the three smallest budgets it concedes at most 8.5% attack success, against the forest's 13.1% and the TCN's 52.2%, and 74.5% of its flows never evade it at any budget against 45.5% for the TCN alone (Table 5.4). As predicted, the meta-learner leans on the RF as the TCN degrades: the forest's share of its verdict rises from zero to 0.53 the moment the attacker spends anything.

RQ2 (purification effectiveness) asked whether the reconstructor moves a flagged flow back onto the benign manifold while emitting a realizable packet sequence. It does, under a condition: scored through the independent NIDS each attack was built to evade, purification recovers 63.0% of the destroyed detection overall, while reconstruction error on the perturbed dimensions falls by $17.0\times$ and the realized packets pass `tshark` 97.2% of the time (Section 5.6). End to end, the gated pipeline cuts the protected NIDS's attack-success rate from 21.1% to 11.6% (Section 5.7.1). In the same position, the diffusion-based purifier of Merzouk et al. [2] recovers 0.5% of the detection lost where TANTGen evaded, against 63.5% for DEPT (Section 5.7.2).

The condition is that the manipulation actually evaded. Where it moved the features without evading, purifying lowers the protected detector's detection, by up to 23 points on `dns_ddos` and 18 on `portscan_iat`, because the denoiser removes signal the NIDS was still using. The aggregate displays large recoveries on five source–mode combinations against regressions on five others, and the gate does not prevent them: it predicts whether a flow is adversarial, not whether purifying it will help. RQ2 is answered affirmatively only for flows a defense would want to purify.

6.3 System Limitations

Six limitations bound the results of Chapter 5. The first is representational, and partly a labelling cost. On the `size` variants of the DNS, NTP, SSDP and LDAP reflection floods, every model scores at or below chance against same-source benign flows, per-attack specialists included. On three of the four, the adversarial tokens are bit-identical to the clean ones, so no detector can tell such a flow from its clean original; on DNS the purifier cuts the footprint without moving DEPT's verdict, so the cause is the representation itself, its 24 prefix statistics and 1088 header bits, not capacity, tuning or training scope. Since these four floods supply about a third of the adversarial test flows, every aggregate in Chapter 5 averages near-perfect and below-chance sub-populations.

The second is that purification without an adequate gate subtracts as well as adds, as RQ2's answer showed; the gate as built barely gates, since purifying every flow yields almost the same end-to-end result.

The third contradicts the system's own premise. The meta-learner is beaten by its own Random Forest on the aggregate streaming axis, widely so on several sources (Section 5.4.1), where a stacking ensemble should be no worse than its best base learner; the carry-forward RF logit reaches the fusion token as one sparsely updated feature that attention under-weights against the dense TCN stream. The fusion still provides the packet-1 verdict and RQ1's robustness, but less than the design hoped for.

The fourth is validation. Purification gain measured through DEPT's own TCN is circular, though the end-to-end claim does not rest on it; each attack is judged by one target NIDS rather than a panel; on `mirai` a target that detects almost nothing even on the clean attack; and the defended full-stream figures run DEPT's models over some flows seen in training. DEPT is also compared with a single published purifier, adapted to its representation, and only against the attacker it was trained on (Section 5.7.2).

The fifth is scope. The purifier corrects all 1090 dimensions without being told which the attacker moved, but the realization step overlays only the two this threat model perturbs (Section 4.5.3), so a flow manipulated in another header field would reach the protected NIDS flagged but unrepaired: "attack-agnostic" describes the detection and the denoiser, not the full deployed pipeline. The feature space is bounded too, to headers, timing, direction, and payload length and entropy, which stay observable as encrypted traffic grows; payload content is never parsed, by design, and may be exactly where the blind spot above lives.

The sixth is efficiency. The tensorization is pure Python, slower than the original C++ nPrint tool it reimplements, setting the throughput floor, and the latency study that would settle the real-time budget of Section 1.2 is left undone.

6.4 Future Work

The limitations above set most of the agenda.

- **A gate that predicts benefit rather than adversariality.** The end-to-end regressions all follow from purifying flows that were still detected. A gate trained on whether purification raises or lowers the protected detector's score, which the system already computes, would convert those losses into no-ops, keeping the gains and raising the overall end-to-end performance.
- **An attack-agnostic realization step.** Deriving the fields to edit from the residual Δ itself and reconciling them against the protocol's constraints, in the spirit of PANTS's SMT-constrained perturbations [44], would generalize the overlay beyond its two hardcoded dimensions.
- **A fusion that never falls below its best base learner.** Re-emitting the RF evidence at every packet without an explicit staleness weight, or letting the meta-learner fall back to the stronger base opinion when the two diverge.
- **More baselines and other attackers.** Repeating the comparison of Section 5.7.2 with other purifiers, and against generators DEPT was not trained on, would show how much of its advantage comes from training on the same attacker.
- **A latency study.** Timing each stage on live traffic, with tensorization moved out of pure Python, would show whether DEPT meets the real-time budget of Section 1.2.
- **What separates a size-mode DNS flood flow from an unedited one.** Which observable quantity, absent from the representation above, distinguishes these flows? Payload content, never parsed by design, is the first place to look.

The direction this thesis considers most significant, though, is **continuous adaptation of the defense**. A static defense, which is what DEPT is once trained, is a fixed target: an adversary that probes it long enough learns its blind spots and routes around it (Chapter 3). Closing the loop means casting the defense as a Reinforcement Learning (RL) agent rewarded by detection outcomes [71–73]: framed as a Markov Decision Process (MDP), it would sit outside the per-packet critical path, watch the ensemble's verdicts and the attacker's drift, and retune against an evolving generator such as TANTGen, measured across attacker generations rather than at one point.

DEPT was built and evaluated on the principle that a defense for learning-based intrusion detection is only worth measuring against an attack that can exist on the wire. Where it fails, it fails for reasons the evaluation identifies rather than hides; it is a defense meant to be built upon, and RL is the natural place to pick up from it.

Bibliography

[1] I. d. A. Pissarra, "TANTGen: Transformer-based adversarial network traffic generator," Master's thesis, Instituto Superior Técnico, Universidade de Lisboa, Lisbon, Portugal, 2026, unpublished Master's Thesis.

[2] M. A. Merzouk, E. Beurier, R. Yaich, N. Boulahia-Cuppens, F. Cuppens, and F. Khomh, "Diffusion-based adversarial purification for intrusion detection," in *Data and Applications Security and Privacy XXXIX: 39th Annual IFIP WG 11.3 Conference (DBSec 2025)*. Springer, 2025, pp. 351–370.

[3] R. Boutaba, M. A. Salahuddin, N. Limam, S. Ayoubi, N. Shahriar, F. Estrada-Solano, and O. M. Caicedo, "A comprehensive survey on machine learning for networking: evolution, applications and research opportunities," *Journal of Internet Services and Applications*, vol. 9, no. 1, pp. 16:1–16:99, 2018.

[4] S. Wang, J. F. Balarezo, S. Kandeepan, A. Al-Hourani, K. G. Chavez, and B. Rubinstein, "Machine learning in network anomaly detection: A survey," *IEEE Access*, vol. 9, pp. 152 379–152 396, 2021.

[5] Z. Xu, Y. Wu, S. Wang, J. Gao, T. Qiu, Z. Wang, H. Wan, and X. Zhao, "Deep learning-based intrusion detection systems: A survey," *arXiv preprint arXiv:2504.07839*, 2025.

[6] F. M. Anis, M. AlAbdullatif, S. Aljbli, and M. Hammoudeh, "A survey on the applications of deep learning in network intrusion detection systems to enhance network security," *IEEE Access*, vol. 13, pp. 185 357–185 373, 2025.

[7] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," in *2nd International Conference on Learning Representations (ICLR 2014)*, 2014.

[8] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.

[9] K. He, D. D. Kim, and M. R. Asghar, “Adversarial machine learning for network intrusion detection systems: A comprehensive survey,” *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 538–566, 2023.

[10] C. Zhang, X. Costa-Pérez, and P. Patras, “Tiki-Taka: Attacking and defending deep learning-based intrusion detection systems,” in *Proceedings of the 2020 ACM SIGSAC conference on cloud computing security workshop*, 2020, pp. 27–39.

[11] F. Pierazzi, F. Pendlebury, J. Cortellazzi, and L. Cavallaro, “Intriguing properties of adversarial ML attacks in the problem space,” in *2020 IEEE symposium on security and privacy (SP)*. IEEE, 2020, pp. 1332–1349.

[12] M. J. Hashemi, G. Cusack, and E. Keller, “Towards evaluation of NIDSs in adversarial setting,” in *Proceedings of the 3rd ACM CoNEXT Workshop on Big DAta, Machine Learning and Artificial Intelligence for Data Communication Networks*, 2019, pp. 14–21.

[13] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, “Practical black-box attacks against machine learning,” in *Proceedings of the 2017 ACM on Asia conference on computer and communications security*, 2017, pp. 506–519.

[14] G. Apruzzese, P. Laskov, and J. Schneider, “SoK: Pragmatic assessment of machine learning for network intrusion detection,” in *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2023, pp. 592–614.

[15] D. H. Wolpert, “Stacked generalization,” *Neural networks*, vol. 5, no. 2, pp. 241–259, 1992.

[16] P. Samangouei, M. Kabkab, and R. Chellappa, “Defense-GAN: Protecting classifiers against adversarial attacks using generative models,” in *International Conference on Learning Representations (ICLR)*, 2018.

[17] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, “Kitsune: An ensemble of autoencoders for online network intrusion detection,” in *Network and Distributed System Security Symposium (NDSS)*, 2018.

[18] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. SciTePress, 2018, pp. 108–116.

[19] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, “Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy,” in *2019 International Carnahan Conference on Security Technology (ICCST)*. IEEE, 2019, pp. 1–8.

[20] I. H. Sarker, “Deep cybersecurity: a comprehensive overview from neural network and deep learning perspective,” *SN Computer Science*, vol. 2, no. 3, p. 154, 2021.

[21] A. Salim, O. Salim, O. M. Khudhur, S. M. Al-Barzinji, and F. M. Jasem, “Feature selection techniques in intrusion detection systems: A review,” *Journal of Cybersecurity and Information Management*, vol. 17, no. 2, pp. 97–112, 2026.

[22] S. Ning, K. Nguyen, S. Bagchi, and Y. Park, “The study of feature engineering in machine learning and deep learning for network intrusion detection systems,” in *2024 Silicon Valley Cybersecurity Conference (SVCC)*. IEEE, 2024, pp. 1–5.

[23] J. Holland, P. Schmitt, N. Feamster, and P. Mittal, “New directions in automated traffic analysis,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2021, pp. 3366–3383.

[24] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, “End-to-end encrypted traffic classification with one-dimensional convolution neural networks,” in *2017 IEEE international conference on intelligence and security informatics (ISI)*. IEEE, 2017, pp. 43–48.

[25] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, and M. Saberian, “Deep packet: A novel approach for encrypted traffic classification using deep learning,” *Soft Computing*, vol. 24, no. 3, pp. 1999–2012, 2020.

[26] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, “Dos and don’ts of machine learning in computer security,” in *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 2022, pp. 3971–3988.

[27] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[28] ——, “Bagging predictors,” *Machine learning*, vol. 24, no. 2, pp. 123–140, 1996.

[29] T. J. Lucas, I. S. De Figueiredo, C. A. C. Tojeiro, A. M. G. De Almeida, R. Scherer, J. R. F. Brega, J. P. Papa, and K. A. P. Da Costa, “A comprehensive survey on ensemble learning-based intrusion detection approaches in computer networks,” *IEEE Access*, vol. 11, pp. 122 638–122 676, 2023.

[30] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” *arXiv preprint arXiv:1803.01271*, 2018.

[31] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu, “WaveNet: A generative model for raw audio,” *arXiv preprint arXiv:1609.03499*, 2016.

[32] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[33] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 633–642.

[34] J. Ghadermazi, S. Hore, A. Shah, and N. D. Bastian, "GTAE-IDS: Graph transformer-based autoencoder framework for real-time network intrusion detection," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 4026–4041, 2025.

[35] S. Ennaji, F. De Gaspari, D. Hitaj, A. Kbidi, and L. V. Mancini, "Adversarial challenges in network intrusion detection systems: Research insights and future prospects," *IEEE Access*, vol. 13, pp. 148 613–148 645, 2025.

[36] P. Sun, S. Li, J. Xie, H. Xu, Z. Cheng, and R. Yang, "GPMT: Generating practical malicious traffic based on adversarial attacks with little prior knowledge," *Computers & Security*, vol. 130, p. 103257, 2023.

[37] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *6th International Conference on Learning Representations (ICLR)*, 2018.

[38] X. Li, W. Sun, H. Chen *et al.*, "ADBM: Adversarial diffusion bridge model for reliable adversarial purification," in *International Conference on Learning Representations (ICLR)*, 2025.

[39] N. Alhussien and A. Aleroud, "AdvPurRec: Strengthening network intrusion detection with diffusion model reconstruction against adversarial attacks," in *2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 2024, pp. 1638–1646.

[40] N. Papernot, P. McDaniel, X. Wu, S. Jha, and A. Swami, "Distillation as a defense to adversarial perturbations against deep neural networks," in *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 582–597.

[41] J. Cohen, E. Rosenfeld, and Z. Kolter, "Certified adversarial robustness via randomized smoothing," in *International Conference on Machine Learning (ICML)*. PMLR, 2019, pp. 1310–1320.

[42] J. Yan, Z. Liu, Y. Xie, S. Liang, L. Liu, and K. Xu, "CertTA: Certified robustness made practical for learning-based traffic analysis," in *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 2025, pp. 7349–7368.

[43] E. Abad Rocamora, G. G. Chrysos, and V. Cevher, “Certified robustness under bounded Levenshtein distance,” in *International Conference on Learning Representations (ICLR)*, 2025.

[44] M. Jin and M. Apostolaki, “Robustifying ML-powered network classifiers with PANTS,” in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 7291–7310.

[45] I. Bibers, O. Arreche, W. Alayed, and M. Abdallah, “Ensemble-IDS: An ensemble learning framework for enhancing AI-based network intrusion detection tasks,” *Applied Sciences*, vol. 15, no. 19, p. 10579, 2025.

[46] Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” *Journal of computer and system sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[47] X. Yang, Y. Zhao, and X. Chen, “A novel transformer-based stacking ensemble method with multi-model integration for cancer classification,” *PeerJ Computer Science*, vol. 11, p. e3314, 2025.

[48] O. Gungor, T. Rosing, and B. Aksanli, “STEWART: Stacking ensemble for white-box adversarial attacks towards more resilient data-driven predictive maintenance,” *Computers in Industry*, vol. 140, p. 103660, 2022.

[49] Z. Yang, L. Li, X. Xu, S. Zuo, Q. Chen, P. Zhou, B. I. P. Rubinstein, C. Zhang, and B. Li, “TRS: Transferability reduced ensemble via promoting gradient diversity and model smoothness,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021, pp. 17 642–17 655.

[50] B. Biggio, I. Corona, G. Fumera, G. Giacinto, and F. Roli, “Bagging classifiers for fighting poisoning attacks in adversarial classification tasks,” in *Multiple Classifier Systems: 10th International Workshop (MCS 2011)*. Springer, 2011, pp. 350–359.

[51] R. Chen, Z. Li, J. Li, J. Yan, and C. Wu, “On collective robustness of bagging against data poisoning,” in *International Conference on Machine Learning*. PMLR, 2022, pp. 3299–3319.

[52] A. Athalye, N. Carlini, and D. Wagner, “Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples,” in *International Conference on Machine Learning (ICML)*. PMLR, 2018, pp. 274–283.

[53] M. Hajizadeh, P. Golchin, E. Nowroozi, M. Rigaki, V. Valeros, S. García, M. Conti, and T. Bauschert, “DeepRed: A deep learning-powered command and control framework for multi-stage red teaming against ML-based network intrusion detection systems,” in *19th USENIX WOOT Conference on Offensive Technologies (WOOT 25)*, 2025, pp. 103–127.

[54] P. Kumar, A. S. M. Tayeen, S. Misra, H. Cao, J. Liu, Q. Gong, and J. Harikumar, “NetDiffuser: Deceiving DNN-based network attack detection systems with diffusion-generated adversarial traffic,” *arXiv preprint arXiv:2603.08901*, 2026.

[55] B. I. Rubinstein, B. Nelson, L. Huang, A. D. Joseph, S.-h. Lau, S. Rao, N. Taft, and J. D. Tygar, "ANTIDOTE: understanding and defending against poisoning of anomaly detectors," in *Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement*, 2009, pp. 1–14.

[56] G. Wan, Z. Shi, W. Huang, G. Zhang, D. Tao, and M. Ye, "Energy-based backdoor defense against federated graph learning," in *The Thirteenth International Conference on Learning Representations*, 2025.

[57] J. Xu, Z. Zhang, and R. Hu, "Detecting backdoor attacks in federated learning via direction alignment inspection," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 20 654–20 664.

[58] P. Rieger, T. Krauß, M. Miettinen, A. Dmitrienko, and A.-R. Sadeghi, "CrowdGuard: Federated backdoor detection in federated learning," in *Network and Distributed System Security Symposium (NDSS)*, 2024.

[59] R. Zhu, D. Tang, S. Tang, X. Wang, and H. Tang, "Selective amnesia: On efficient, high-fidelity and blind suppression of backdoor effects in trojaned machine learning models," in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 682–700.

[60] K. Kumari, P. Rieger, H. Fereidooni, M. Jadliwala, and A.-R. Sadeghi, "BayBFed: Bayesian backdoor defense for federated learning," in *2023 IEEE symposium on security and privacy (SP)*. IEEE, 2023, pp. 737–754.

[61] D. T. Nguyen, N. N. Tran, T. T. Johnson, and K. Leach, "PBP: Post-training backdoor purification for malware classifiers," in *Network and Distributed System Security Symposium (NDSS)*, 2025.

[62] F. Alotaibi and S. Maffeis, "Mateen: Adaptive ensemble learning for network anomaly detection," in *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses*, 2024, pp. 215–234.

[63] M. Zhang, K. Bi, W. Chen, Q. Chen, J. Guo, and X. Cheng, "CausalDiff: Causality-inspired disentanglement via diffusion model for adversarial defense," *Advances in Neural Information Processing Systems*, vol. 37, pp. 124 791–124 819, 2024.

[64] Z. Li, H. Zhou, M. Rei, and L. Specia, "DiffuseDef: Improved robustness to adversarial attacks via iterative denoising," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 9259–9274.

[65] V. Kumar, K. Kumar, and M. Singh, "NIDS-CBAD: detecting adversarial attacks in network intrusion detection systems using domain constraints," *International Journal of Machine Learning and Cybernetics*, vol. 16, no. 7-8, pp. 4213–4234, 2025.

[66] O. Olukola and N. Rahimi, “AMDS: Attack-aware multi-stage defense system for network intrusion detection with two-stage adaptive weight learning,” *arXiv preprint arXiv:2603.00859*, 2026.

[67] N. Carlini, A. Athalye, N. Papernot, W. Brendel, J. Rauber, D. Tsipras, I. Goodfellow, A. Madry, and A. Kurakin, “On evaluating adversarial robustness,” arXiv, Tech. Rep. arXiv:1902.06705, 2019.

[68] F. Tramèr, N. Carlini, W. Brendel, and A. Madry, “On adaptive attacks to adversarial example defenses,” in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020, pp. 1633–1645.

[69] F. Croce and M. Hein, “Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks,” in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 119, 2020, pp. 2206–2216.

[70] M.-I. Nicolae, M. Sinn, M. N. Tran, B. Buesser, A. Rawat, M. Wistuba, V. Zantedeschi, N. Baracaldo, B. Chen, H. Ludwig, I. M. Molloy, and B. Edwards, “Adversarial robustness toolbox v1.0.0,” arXiv, Tech. Rep. arXiv:1807.01069, 2018.

[71] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[72] T. T. Nguyen and V. J. Reddi, “Deep reinforcement learning for cyber security,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 8, pp. 3779–3795, 2023.

[73] G. Caminero, M. Lopez-Martin, and B. Carro, “Adversarial environment reinforcement learning algorithm for intrusion detection,” *Computer Networks*, vol. 159, pp. 96–109, 2019.